

Towards Horizontally Scalable Kubernetes Controllers

STUDIENARBEIT

für die Prüfung zum
Master of Science (M.Sc.)

im Studiengang
Informatik

am Center for Advanced Studies
der Dualen Hochschule Baden-Württemberg

von

Tim Ebert

Matrikelnummer	8559152
Bearbeitungszeitraum	27.06. - 26.12.2022
Dualer Partner	SAP SE / Schwarz IT KG
Prüfer	Prof. Dr.-Ing. habil. Dennis Pfisterer

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Studienarbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Aus fremden Quellen direkt oder indirekt übernommene Gedanken habe ich als solche kenntlich gemacht. Die Arbeit habe ich bisher keinem anderen Prüfungsamt in gleicher oder vergleichbarer Form vorgelegt. Sie wurde bisher auch nicht veröffentlicht.

Ich erkläre weiterhin, dass ich ein unverschlüsseltes digitales Textdokument der Arbeit (in einem der Formate .doc, .docx, .odt, .pdf, .rtf) in Moodle hochgeladen habe. Alle digital eingereichten Exemplare entsprechen in Inhalt und Wortlaut ausnahmslos der gedruckten Ausfertigung.

Ort, Datum

Unterschrift

Abstract

Controllers play an essential role in every Kubernetes cluster by realizing the desired state specified declaratively in API objects – a process referred to as reconciliation. To prevent uncoordinated and conflicting actions of concurrent reconciliations, controllers use a leader election mechanism to determine a single active controller instance. Because of this, reconciliations cannot be distributed among multiple controller instances which effectively limits the system’s capacity and throughput by the available machine sizes and network bandwidth of the underlying infrastructure. I.e., leader election prevents Kubernetes controllers from being horizontally scalable.

This thesis presents the first steps towards horizontally scalable Kubernetes controllers by introducing a design for distributing the reconciliation of API objects across multiple controller instances, i.e., sharding for Kubernetes controllers. It shows that proven sharding mechanisms used in distributed databases can be applied to the problem space of Kubernetes controllers as well to overcome the mentioned scalability limitation. The proposed design includes lease-based membership and failure detection as well as consistent hashing for partitioning. Kubernetes API machinery primitives – namely, labels and label selectors – facilitate coordination between controller instances and prevent concurrent reconciliations of individual objects by multiple instances.

For demonstration and evaluation, the sharding design is implemented based on the controller-runtime library and used in an example operator. Systematic load test experiments show that the sharding implementation achieves a good distribution of object responsibility and resource usage across individual shard instances. However, the experiment results also point out that one of the controller instances always has a high resource footprint in comparison to the other shards which limits the scalability of the system. Nevertheless, future work can perform optimizations of the presented design to overcome the discovered flaw.

The presented design and implementation allow scaling Kubernetes controllers horizontally providing potential for a wide range of applications in the Kubernetes ecosystem.

Contents

1	Motivation	1
2	Background	2
2.1	Architecture	2
2.2	Kubernetes Resource Model	3
2.3	API Machinery	5
2.4	API Extensions	8
2.5	Controllers	9
2.6	Leader Election	12
2.7	Sharding in Distributed Databases	13
3	Requirement Analysis	16
3.1	Current Scalability Limitations	16
3.2	Requirements for Horizontal Scalability	18
4	Design	21
4.1	Architecture	21
4.2	Membership and Failure Detection	22
4.3	Partitioning	23
4.4	Coordination and Object Assignment	24
4.5	Preventing Concurrency	25
5	Implementation	27
5.1	Webhosting Operator	27
5.2	Architecture	30
5.3	Shard Lease	32
5.4	Lease Controller	33
5.5	Sharder Controllers	33
5.6	Object Controller	35
6	Evaluation	37
6.1	Deployment	37
6.2	Monitoring	38
6.3	Experiment	41
6.4	Results	43
6.5	Discussion	45

7 Conclusion and Future Work	47
Appendix	V
List of Figures	VI
List of Tables	VII
List of Code	VIII
References	IX

1 Motivation

In Kubernetes, the desired and actual state of the distributed system are stored centrally in etcd [1] and exposed by the Kubernetes API server. The logic for turning the desired state into the actual state of the system (reconciliation) is implemented in dedicated components called controllers. Storing the desired and actual state is the API server's responsibility and reconciling both is the task of controllers. Because these tasks are decoupled Kubernetes is an asynchronous and eventually consistent [2], [3] system. [4]

Kubernetes controllers as such are stateless components, they persist the state they work on in external systems – in etcd via the API server. I.e., the system doesn't lose any important state if controllers fail or restart. However, Kubernetes controllers typically use a leader election mechanism to determine a single active controller instance (leader). When deploying multiple instances of the same controller, there will only be one active instance at any given time, other instances will be on standby. This is done to prevent controllers from performing uncoordinated and conflicting actions.

If the current leader stops or loses leadership (e.g., during a rolling update or network partition) another instance takes over leadership and becomes the active instance. Such a setup can be described as an “active-passive high-availability (HA) setup” [5]. It minimizes controller downtime and facilitates fast failovers. However, it cannot be considered horizontal scaling because work is not distributed across multiple instances [6], [7]. Because of this, Kubernetes controllers are not horizontally scalable.

This restriction imposes scalability limitations for Kubernetes controllers. I.e., the rate of reconciliations, amount of objects, etc. are limited by the available machine size and network bandwidth of the underlying infrastructure. In contrast to usual stateless applications, one cannot increase the capacity and throughput of the system by adding more instances (horizontal scaling) but only by using instances with more resources (vertical scaling).

This thesis explores approaches for distributing the reconciliation of Kubernetes objects across multiple controller instances. It attempts to lift the restriction of having only one active instance per controller. For this, mechanisms are required for determining which instance is responsible for which object to prevent conflicting actions. The thesis evaluates how proven mechanisms for sharding and partitioning from the field of distributed databases [8], [9] can be applied to this problem.

2 Background

2.1 Architecture

Kubernetes is an open-source container orchestration system that manages containers [10] on a cluster of machines. It is a distributed system composed of several components, that can be categorized as control plane components and worker node components. The Kubernetes control plane consists of etcd and API server for storing cluster state and a set of controllers for managing the cluster’s worker nodes and the workload itself. Each worker node hosts a cluster agent called kubelet, a container runtime, and other system components that are responsible for tasks like networking, monitoring, etc. [4]

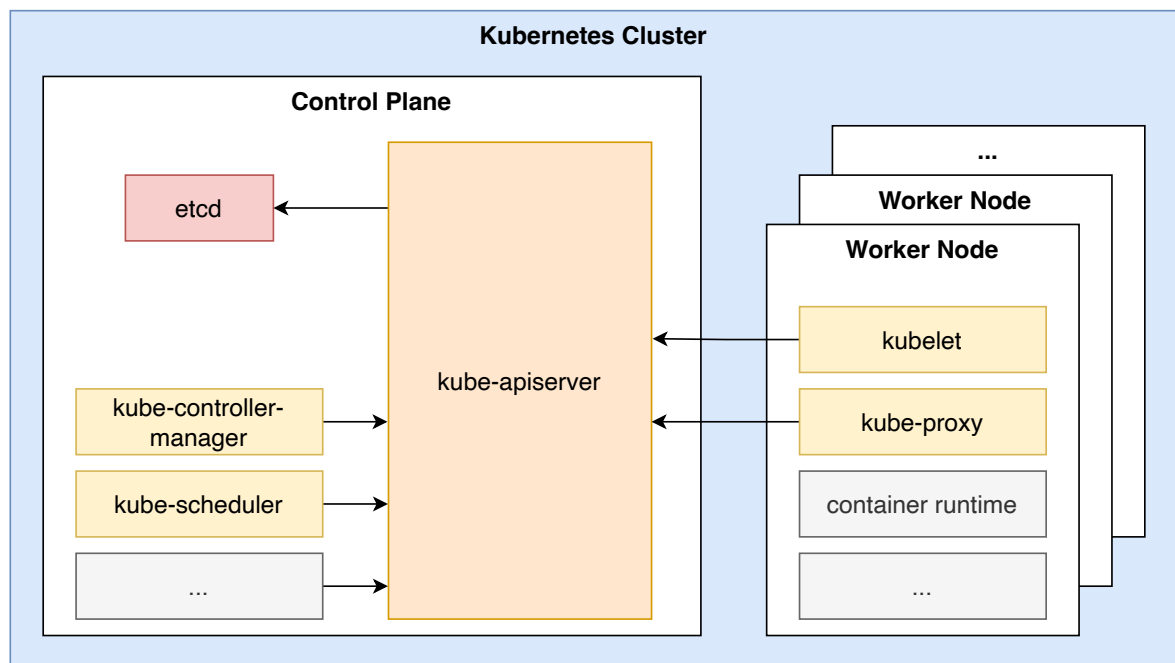


Figure 1: Kubernetes cluster architecture [4]

etcd is the only stateful component of the Kubernetes cluster architecture. It is a consistent key-value store, that is used by the API server for storing the entire cluster state. etcd itself can be run as a set of etcd instances (also called “cluster”) for ensuring high availability. Running multiple instances of etcd doesn’t increase performance though, as there will only be one leader accepting write operations. The only component communicating directly with etcd is the API server, all other components use etcd only indirectly through the API server. [1], [4]

The Kubernetes **API server** exposes a REST API [11] – the Kubernetes API – as the single point of entry for all cluster operations. As a stateless component, it can be scaled horizontally in order to increase throughput and availability by load balancing between multiple instances [6]. The cluster’s state is persisted and managed in form of API objects following the “Kubernetes Resource Model” (sec. 2.2). The API server allows retrieving and manipulating objects via standard HTTP mechanisms using the respective API endpoints. It transforms API objects from and to the storage format that is used for persisting them in etcd. For human users of the API, objects are typically presented in YAML. The operations on objects and related mechanics that the API server exposes are often referred to as “API machinery” (sec. 2.3).

Kubernetes **controllers** control the actual state of the cluster to match the desired state, that the user or other controllers specified declaratively. Controllers work on individual API objects retrieved from the API server and implement their actual business logic. For this purpose, controllers use the API server’s mechanics for listening for changes to API objects and update the current state of the objects on the API server. Typically, there is a single controller for each kind of API object. Controllers are contained in multiple components of the cluster, most prominently in the Kubernetes controller manager and scheduler. These components contain most of the controllers for implementing the logic of Kubernetes’ core API objects and are essential parts of the control plane. [12]

kubelet and **kube-proxy** are node components running on every machine of the cluster. They instrument the container runtime and networking stack for managing the cluster’s workload. At the core, both components are controllers as well and use the same mechanics as controllers that are part of the control plane. However, kubelet, kube-proxy, and other node components as such are not relevant for this thesis and thus not discussed in more detail.

2.2 Kubernetes Resource Model

Kubernetes is an API-centric system, meaning all users and components interact with the same central API, i.e., there is no internal or hidden API. All resources of the Kubernetes API follow the same patterns, which are also referred to as the Kubernetes Resource Model. [13]

API objects are managed declaratively rather than imperatively. This means users declare the desired state of objects via the respective API resources instead of performing

iterative actions on them. Changes to the desired state are accepted instantly without waiting for them to manifest in the system. Controllers pick up the declared specification asynchronously and update the status of API objects while the actual state gradually converges with the desired state.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: default
  annotations:
    foo: bar
  labels:
    app: nginx
  uid: f4dc8f2d-075d-41b1-a511-0f44671cd5d0
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
status:
  availableReplicas: 1
  readyReplicas: 1
  replicas: 1
  updatedReplicas: 1
```

Code 1: Example API object

All API objects contain common metadata (`apiVersion` , `kind` , `metadata`) describing general object information. `apiVersion` and `kind` fully identify the object's type in a given API group and the version it is described in (`GroupVersionKind`). The `metadata` section contains the object's identifying `name` along with user-defined key-value pairs as `labels` and `annotations` . Labels are used for filtering and grouping API objects by certain attributes, while annotations allow adding arbitrary information. API objects are grouped by user-created namespaces (specified in `metadata.namespace`), if the respective type is namespace-scoped. Additionally, all

API objects are assigned a globally unique identifier in `metadata.uid`. As it changes when an object is deleted and recreated, it can be used to distinguish between different instances of the same kind in the same namespace with the same name over time.

API objects can specify so-called owners in the `metadata.ownerReferences` field. This is used by controllers for efficiently mapping objects to their owners. Furthermore, an API object is automatically deleted by the garbage collector controller once all referenced owners are gone.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-8f458dc5b
  namespace: default
  # ...
  ownerReferences:
  - apiVersion: apps/v1
    blockOwnerDeletion: true
    controller: true
    kind: Deployment
    name: nginx
    uid: f4dc8f2d-075d-41b1-a511-0f44671cd5d0
```

Code 2: Example owner reference

Most API objects also contain `spec` and `status` sections. The `spec` section contains the user-declared desired state of the object, while the `status` section contains the state observed by the responsible controller. [4]

2.3 API Machinery

The API server offers a discovery API (path `/api` and `/apis`) which allows clients to discover available API groups, resources, and versions. Additionally, this API presents mappings between fully-qualified API kinds – `GroupVersionKind` – and their fully-qualified API resources – `GroupVersionResource`. This process is also referred to as “REST mapping” and is used by clients to build the API URL corresponding to the API object they want to interact with. I.e. the API URL of a given object follows this pattern: [13]

```
/<prefix>/<group>/<version>/namespaces/<namespace>/<resource>/<name>
```

The path prefix is `/apis`, except for resources belonging to the `core` API group, which is signified by a missing group in the `apiVersion` field, in which case the prefix `/api` is used.

Resources can have so-called subresources, which offer specific actions on a given API object. E.g., the `/status` subresource allows updating the `status` section of an object, which enables segregating authorization policies for users and controllers. Subresources are available under the API URL of the targeted object, suffixed with the subresource name.

API objects can be requested and manipulated in different API versions, which also describe the maturity level of the API. However, the API server stores all objects only once in a designated API version. If a client retrieves or updates objects in a version that is different from their storage version, the API server converts them back and forth. Different API versions can thus be considered as different representations of the same API object, which allows Kubernetes to guarantee backward-compatibility of the API. This is particularly important for the distributed architecture of Kubernetes and allows evolving and upgrading components independently.

Requests to API resources have different types – so-called verbs – that are identified by the used HTTP verb, path, and query parameters. `get` requests use HTTP `GET` and a full API URL for retrieving a single object. `list` requests are `get` requests but omit the name for retrieving all objects of a given kind in the cluster. `list` requests can be restricted to a namespace if the resource is namespace-scoped. `list` requests including the query parameter `watch=true` are called `watch` requests, which stream change events for a given kind of object. These verbs are categorized as non-mutating. The mutating verbs – not described in further detail – are: `create`, `update`, `patch`, `delete`, and `deletecollection`. [4]

Every Kubernetes object carries a `metadata.resourceVersion` field, which uniquely identifies the object's internal version as stored in etcd. All mutating operations change the `resourceVersion` field, allowing clients to detect when an object was changed. Currently, the field is backed by the revision that etcd stores as metadata for all keys and returns on all requests [1]. Resource versions are represented as strings and must be treated as opaque by clients, i.e., they may only be checked for equality. In Kubernetes, the resource version is used for implementing optimistic concurrency control and efficient detection of changes (watches). In addition to the `resourceVersion` field, most API objects carry a `metadata.generation` field. The `generation` field is set to `1` on creation and increased on every change to the `spec` section and on deletion. This field is typically used by controllers for filtering out irrelevant change events. Ad-

ditionally, controllers typically report the latest observed generation of the API object in `status.observedGeneration` allowing other clients to determine if changes to the desired state have already been picked up by the controller.

For optimistic concurrency control, clients typically pass the resource version when updating or patching objects as part of a read-modify-write loop. With this, the API server checks the given resource version against the resource version of the current object in the store. If these values don't match, another concurrent update has been successfully written and the request is denied with `409 Conflict`. This is an important mechanism in Kubernetes that prevents controllers and other clients from performing uncoordinated and conflicting changes to API objects. [14]

Watch requests are an important factor for the scalability of Kubernetes. Watches are long-lived requests which stream notifications for changes to API objects, that clients care about. Typically, clients make an initial `list` request for the collection of objects they are interested in and start listening for changes since the retrieved version. For this, clients pass the resource version from the retrieved list as a query parameter of the watch request. Starting from the specified resource version, the API server sends notifications in form of JSON documents affecting the watched objects with one of the types `ADDED`, `MODIFIED`, and `DELETED`. The watch mechanism internally relies on etcd's `Watch` service [1]. However, the API server only opens a single watch connection to etcd per resource type and dispatches events to all watches registered by API clients. This is done to reduce the request load on etcd and save encoding/decoding effort. Additionally, the API server keeps a history of watch events in memory – the “watch cache” – allowing clients to lag behind and to catch up with changes since an older resource version [4].

Label selectors are frequently used in Kubernetes to select a group of objects of the same kind based on common attributes stored in form of labels. Clients can supply them via the `labelSelector` query parameter of `list` and `watch` requests (as well as `deletecollection` requests). This instructs the API server to return a filtered list of objects that match the given selector, which saves encoding/decoding effort as well as network transfer. However, it's important to note that the API server itself issues unfiltered requests to etcd and filters the results in memory before sending the result list back to clients. This means the given label selector is matched on each list item or watch event individually.

Field selectors are similar to label selectors but filter by matching individual fields of objects rather than labels, e.g. `Pod.spec.nodeName`. However, field selectors cannot be used for selecting arbitrary object fields. Instead, they can only be used with specific

fields for which the API server supports field selectors. For `list` request, field selectors behave similarly to label selectors, meaning they are processed for each list item before returning the filtered result to clients. Under certain circumstances, field selectors can however reduce processing effort on the API server side for `watch` requests via index lookups.

2.4 API Extensions

The Kubernetes API server features several mechanisms for extending the API and integrating third-party components. These include most importantly: webhooks and custom resources. Webhooks can be registered via dedicated API objects and provide means to validate or mutate API requests. However, webhooks are not discussed in further detail in this thesis. Custom resources, on the other hand, offer options to augment the set of built-in API resources with additional resources that follow the same API semantics. [4]

There are two ways to register custom resources with the API server: via `APIServices` (often referred to as API aggregation) and `CustomResourceDefinitions` (CRDs). An `APIService` is a usual API object that configures the API server to delegate API requests for a certain set of `GroupVersions` to another API server (extension API server), that is typically deployed as a dedicated component into the cluster. This extension API server needs to implement the same semantics as the Kubernetes API server itself so that clients can use the extended resources by the same means as built-in resources. From a client's perspective, custom resources are available at the same endpoint (via the Kubernetes API server) and follow the same characteristics as described in secs. 2.2, 2.3. Extension API servers are developed, deployed, and managed independently of the Kubernetes API server and thus feature full flexibility but also require a certain amount of development and operations effort.

In contrast to that, CRDs offer a lightweight mechanism for augmenting the Kubernetes API with custom resources, though with less flexibility. `CustomResourceDefinitions` are API objects that each register a single custom resource with the API server in a purely declarative manner. Most importantly, they specify the API group, versions, and kind along with an optional API schema. As soon as CRDs are created, the Kubernetes API server starts serving endpoints for the specified resources. Although the API server doesn't know the concrete structure and meaning of the resources, it offers the same characteristics for such API objects. This includes the same object structure (metadata, spec, and status), the same API request semantics (including

watch requests), discovery endpoints, label selectors, and so on. [4], [12]

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: websites.webhosting.timebertt.dev
spec:
  group: webhosting.timebertt.dev
  names:
    kind: Website
    listKind: WebsiteList
    plural: websites
    singular: website
  scope: Namespaced
  versions:
  - name: v1alpha1
    served: true
    storage: true
    subresources:
      status: {}
    schema:
      openAPIV3Schema:
        description: Website enables declarative management of hosted websites.
        # ...
```

Code 3: Example CustomResourceDefinition

By leveraging these mechanisms, the Kubernetes API can be extended to provide declarative management of arbitrary API resources. However, custom resources as such don't implement any corresponding business logic. For this, additional controllers can be deployed that implement the logic for realizing the desired state declared by the API objects. This combination of a custom resource and a custom controller is often referred to as the “operator pattern”, where the custom controller is referred to as the “operator”. Operators can be used to codify any kind of operational knowledge for applications and represent a powerful mechanism for running cloud-native workloads on Kubernetes. [12]

2.5 Controllers

Kubernetes controllers are composed of the following building blocks: cache, event handlers, work queue, and worker routines. Most of them are implemented in libraries, mainly client-go, the official Kubernetes API client for the go programming

language. Theoretically, controllers can be implemented in any arbitrary programming language. However, most controllers use go to benefit from the matured and performance-optimized libraries that the official Kubernetes controllers are based on as well. [15]

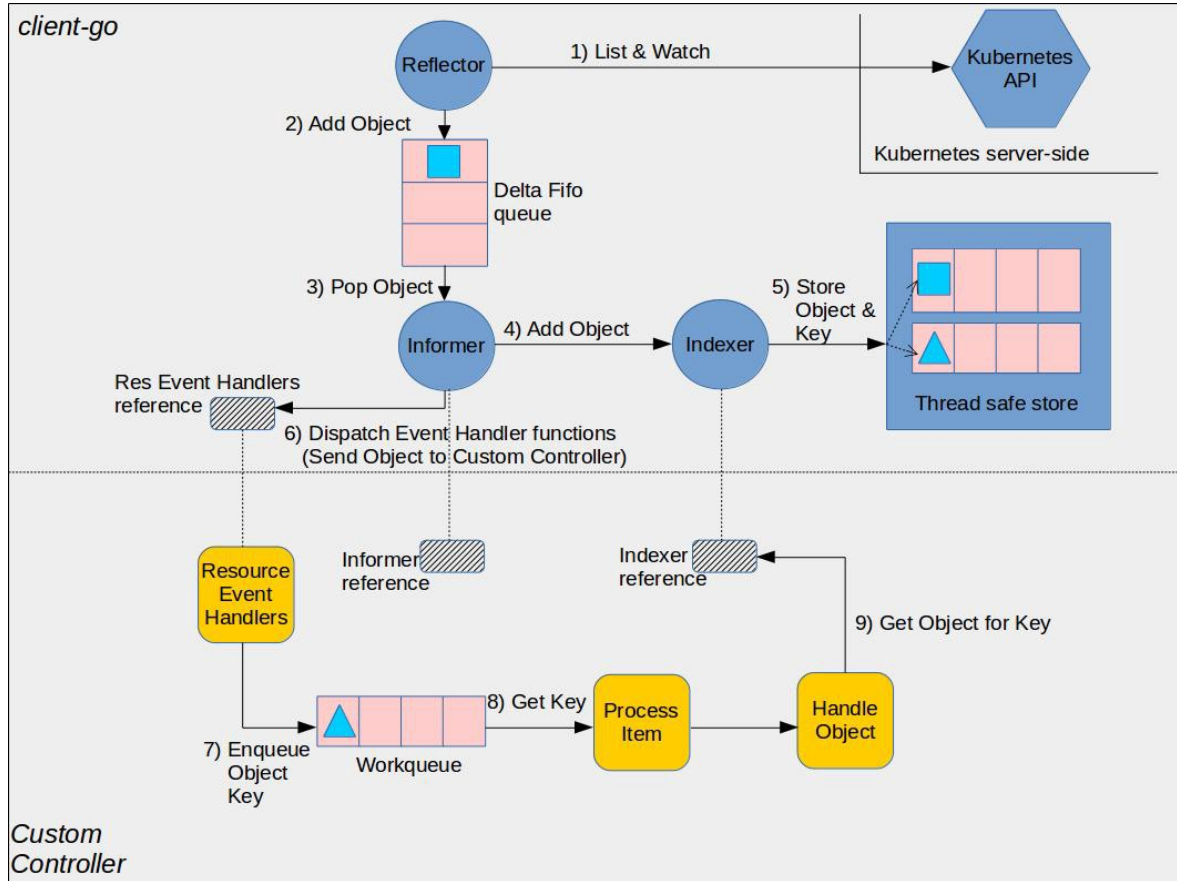


Figure 2: Building blocks of a controller [15]

A controller's **cache** is responsible for watching the controller's object type on the API server, informing the controller about changes to the objects, and making the objects available to the controller in memory in form of an indexed store. Therefore, it starts a so-called reflector that lists and watches a given object type as described in sec. 2.3. The reflector emits corresponding delta events and adds them to a queue. An informer then reads these events from the queue and updates changed objects in the store accordingly for later retrieval by the controller. The store (indexer) is a flat key-value store with additional indices for increasing the performance of namespaced lookups or lookups with field selectors, that the controller frequently uses. In addition to saving objects in the store, the informer also distributes notifications to all event handlers registered by the controller. Note, that the store is not a write-through cache, meaning controllers might read an old version of the object they modified from the

cache until the watch connection receives the respective change event.

Controllers can use caches for multiple object types if they work with different kinds of objects during reconciliation or listen for changes to related objects. Therefore, controller caches are typically shared between all controllers of a single binary in order to reduce processing effort and memory consumption (`SharedIndexInformer`). Caches can also be configured to use filtered list and watch requests (e.g., by namespace or label) to reduce overhead for processing, transferring, and storing objects that controllers are not interested in.

Controllers add **event handlers** to informers for notifications about all important watch events. Typically, event handlers perform basic filtering for relevant changes based on the changed object to determine whether the controller needs to act on it or not. This is done to reduce unnecessary reconciliation work. If work needs to be done on the watched object, event handlers add the object's key (`namespace/name`) to the work queue. However, event handlers don't always enqueue a key for the changed object itself. Instead, they might also perform a mapping between watched objects and objects that the controller is responsible for. For example, the `Job` controller manages `Pods` for carrying out the actual work and adds an `ownerReference` to the owned `Pods` . When a `Pod` completes its work, the `Job` controller is notified of the status change via a watch event and enqueues the owning `Job` designated in `Pod.metadata.ownerReferences` to update the `Job` object's status accordingly.

A controller's **work queue** centrally keeps track of all keys of objects that the controller needs to perform actions on. It decouples event handling and actual reconciliation: controllers might only act upon objects once taken from the queue, but never in event handlers. The work queue also keeps track of all objects that are currently being processed by worker routines. It ensures, that a given object key is only picked up by a single worker at a time, even if the key is added to the queue multiple times. It only emits a key again once the processing worker has marked the key as processed. Furthermore, the work queue implements multiple mechanisms that are important to the controller's stability and scalability. For example, if work on a given object fails because of some error, the object's key is re-queued, so that it is picked up by a worker again. The work queue keeps track of retries and applies an exponential backoff strategy and jittering in order to break the periodicity of the system. Additionally, it applies overall and per-item rate limiting to the key emission rate to protect the controller from load spikes.

Eventually, the controller's actual work is carried out by **worker routines**, also referred to as "control loops". Note, however, that they are not looping over a single

object as often depicted. Instead, they react to relevant changes as notified by the watch connection and pick a single key from the work queue at a time. Workers mark the key as being processed, retrieve the full object from the cache and finally carry out the actual business logic of the controller. If implemented in go, workers run as concurrent goroutines. The number of concurrent workers reading from the queue and performing work can be raised in order to increase throughput and decrease queue wait time.

Controllers take the desired state of objects, observe the actual state and perform actions to reconcile both. Generally, controllers need to be level-driven, meaning their actions should be based on observations of the full state independent of any intermediate state changes. However, controllers are typically edge-triggered for increasing the scalability of the system and reducing unnecessary work and network transfer. Therefore, they use API server watches instead of long-pulling. [4], [12]

2.6 Leader Election

There might be multiple instances of a single controller binary, e.g., during rolling updates or in HA setups. To prevent conflicting actions of multiple instances, one instance is elected to be the active one – the leader. Controllers may only perform reconciliations when the instance is the current leader. The leader election mechanism follows a simple protocol based on Kubernetes API objects. For this purpose, dedicated `Lease` objects are used, `Endpoints` and `ConfigMaps` were used historically. In these objects, a leader election record is persisted (code 4), which states the current leader as well as when the lease has been acquired and for how long it is valid.

All instances carry a unique identity, composed of pod name and container ID or any other unique identifier for the instance's process. If there currently is no active leader, all instances try to create or update the respective object to become the leader. As the API server denies concurrent writes to the same object (optimistic concurrency control, sec. 2.3), only a single write request can be successful, which determines the elected leader. Once a given instance has successfully acquired leadership, it regularly renews its leadership by updating `renewTime`. As long as the active leader renews the lease before `leaseDurationSeconds` expires, it continues to perform reconciliations and other instances need to stay on standby. If however, the leader fails to renew its lease in time (loss of leadership), it must stop performing any actions and a new leader is elected. In an HA setup with multiple instances, this mechanism ensures fast failovers in case of losing the active leader.

```
apiVersion: coordination.k8s.io/v1
kind: Lease
metadata:
  creationTimestamp: "2022-06-16T08:54:48Z"
  name: kube-controller-manager
  namespace: kube-system
  resourceVersion: "522"
  uid: 562f1ebe-4da1-4dda-80ce-63b9a50337ab
spec:
  acquireTime: "2022-06-16T08:54:48.860913Z"
  holderIdentity: test-control-plane_2629b027-995b-40fe-991e-7aa3770bb654
  leaseDurationSeconds: 15
  leaseTransitions: 0
  renewTime: "2022-06-16T08:56:17.146876Z"
```

Code 4: Example Lease

2.7 Sharding in Distributed Databases

This section summarizes important and proven sharding approaches in well-known distributed databases [8], [9]. It is structured by mechanisms that are common across different implementations. Chapter 4 evaluates which of these mechanisms and approaches can be applied to Kubernetes controllers as well.

First, nodes of distributed database clusters need to discover their peers and derive information about their state, e.g. whether they are ready to host data and serve queries or not. One common approach for this **membership mechanism** is gossip-based and used for example in Cassandra and CockroachDB [16], [17]. In this approach, all nodes of the system are equal and there is no designated leader or central instance. Nodes announce themselves to the cluster and discover their peers by communicating with the cluster. Node information is then propagated to other nodes via ongoing communication between peers, which is called gossip. Several nodes can be used as seed nodes, which are used to bootstrap the gossip protocol. If a node failure occurs the other nodes independently detect the failure based on the gossiped information.

Bigtable implements a different approach to discovering membership information, which is based on leases in a central metadata store [18]. Here, individual nodes don't communicate with each other for exchanging membership information but instead announce themselves to the master node by acquiring individual locks in Chubby, a highly-available lock service [19]. As long as an instance is alive, it renews its lock lease to signal readiness to the master and serves data to clients. The lock is deleted

on termination to inform the master about the voluntary disruption. The master watches the servers directory in Chubby and asks instances for the lock status to discover ready instances. If an instance fails to report a healthy lock, the master tries to acquire the instance's lock to ensure Chubby is reachable. If this succeeds, the master has successfully detected the instance failure and deletes the corresponding lock in Chubby so that the instance will not serve data again. Another approach for implementing the membership mechanism includes frequent communication between a master and individual instances for checking their health status as seen in Spanner [20].

After discovering available instances, sharded databases need an **algorithm for partitioning data**, i.e., for determining which data is stored by which instance. For this, each unit of data holds a partition key that is used as input for the sharding algorithm. Partition keys are typically backed by a physical index and often are part of a primary key structure. In many database systems, application developers can designate a certain column as the partition key which allows co-locating related data. If there is a natural sharding key in the applications business model, it can be used for example to place all data belonging to the same workspace, project, or similar structures on the same servers for improved query performance. While the concept of partition keys is common among distributed databases, there are different approaches for the actual partitioning algorithm that assigns subsets of data to instances. [9]

Bigtable, MongoDB, CockroachDB, and Spanner for example have some form of master instance to control data partitioning using some arbitrary logic. In these systems, a central instance decides where to place data according to current server utilization, placement constraints, and similar indicators. The partitioning information is persisted in metadata tables or metadata key ranges respectively that are known to all instances. This allows all servers to discover where a given data subset is located for querying. [17], [18], [20], [21]

On the other hand, Cassandra and Dynamo use consistent hashing as a partitioning algorithm [22] – also known as ring hashing. In consistent hashing, servers and partition keys are both hashed onto an imaginary ring. Data is assigned to the server with the key that has the smallest hash larger than the hash of the partition key. When new servers are added to the ring or existing servers are removed from a ring with n servers, only $1/n$ of all entries are moved to another server on average. This provides near-optimal stability of partitioning in dynamic environments during scale-out/in operations. A common improvement of consistent hashing is to use so-called virtual nodes for achieving a better distribution of keys in clusters with a small number of members.

For this, each instance is mapped to multiple tokens that are hashed onto the ring. [16], [23], [24]

The third important sharding mechanism in distributed databases is the **coordination of client requests** and the **storage of assignments**. Consistent hashing is deterministic given the partition key and cluster members' statuses. Hence, there is no need to store data assignments in a central place in Cassandra and Dynamo. Instead, data locality can be determined by all nodes independently, e.g., based on the gossiped membership information. In Cassandra and Dynamo, clients can send requests to any cluster node. The node then takes care of determining the responsible server for the queried data and proxies the request accordingly [16], [24]. Similarly, clients can also send requests to any cluster member of a CockroachDB cluster, which looks up and caches data locality from the metadata key range and proxies them to the responsible node accordingly [17]. In contrast to that, Bigtable stores the location of its metadata table in Chubby, which clients need to request to determine data locality. For each request, clients first locate the requested data from the metadata table or possibly from a cache and then send the request to the responsible server [18]. In MongoDB and Spanner, there are dedicated server instances that don't store data and only respond to requests. These proxies are responsible for locating data in the cluster and caching that information for further client requests [20], [21]. One important aspect of all approaches to request coordination in distributed databases is that there is no single point of failure or bottleneck on the request path, that all client communication goes through.

Last, many sharded database setups also replicate data subsets to provide high availability and durability. Typically, there is some form of **concurrency control** involved to provide consistency despite concurrent requests to different replicas of the same data subset. Depending on the database's consistency guarantees, different approaches are applied. Cassandra and Dynamo use timestamp versioning per data item and resolve conflicts by applying the "last write wins" policy [16], [24]. In other database systems, a form of distributed consensus algorithm is leveraged to provide concurrency control. Examples of this are Paxos as used in Chubby [19], raft as used in etcd [1], or multi-group derivatives of them as used in Spanner and CockroachDB [17], [20].

3 Requirement Analysis

Based on the motivation for this study project and the described background, this chapter first describes the current limitations in scaling Kubernetes controllers. Afterward, it analyzes what is required to make Kubernetes controllers horizontally scalable.

3.1 Current Scalability Limitations

Kubernetes controllers need to prevent uncoordinated and conflicting actions from different instances on the same API objects. Therefore, controllers currently use leader election mechanisms to determine a single active instance at any given time. Even if multiple instances are running at the same time, only a single instance carries out the actual work – the current leader. This means, that a controller’s work cannot be distributed across multiple controller instances. In the context of this thesis, such a setup is referred to as a “singleton controller” setup.

The most important scalability dimensions of a Kubernetes controller are capacity and throughput. The capacity of a controller shall be defined as the total number and size of API objects existing in the cluster simultaneously that are watched by the controller. Increasing a controller’s capacity is mainly reflected in increased cache size. When reaching the upper limit of the controller’s capacity, it might run out of memory and stop working entirely. This sets hard limits for how many API objects can be managed in a single cluster and might limit the ability to support increased usage of the service that the controller is responsible for. A controller’s throughput shall be defined as the rate of reconciliations it performs and watch events it handles. If API objects are created, updated, and deleted frequently the controller’s throughput needs to be high enough to properly handle all reconciliation requests. When the rate of reconciliation requests exceeds the controller’s maximum throughput, the system will become less responsive as reconciliation requests are queued up and delayed. This results in a decreased user experience and stability of the system.

Because a controller’s work cannot be distributed, its capacity and throughput can only be increased by scaling it vertically, i.e., by adding more resources to the instance. However, one cannot increase capacity or throughput by adding more controller instances. I.e., as of now, Kubernetes controllers are not horizontally scalable. To understand what is required for scaling Kubernetes controllers horizontally, it’s important to note

which resource dimensions are relevant for scaling. Generally speaking, the following resource requirements increase with the controller’s capacity and throughput:

resource	depends on
CPU	rate and size of API objects encoded, decoded (API requests and watch events), rate and CPU consumption of actual reconciliations
memory	number and size of API objects stored in the controller’s caches
network bandwidth	rate and size of API requests and watch events

Table 1: Resource requirements of a Kubernetes controller

As controllers can only be scaled by deploying larger instances, this effectively limits their scalability by the available machine sizes and network bandwidth. Note, that using bigger machines and broader network connections typically harms cost-efficiency at the top end of the spectrum. Hence, it is desirable to make controllers horizontally scalable and rather distribute multiple instances across smaller machines instead of deploying bigger instances.

Additionally, increasing a controller’s capacity and throughput also increases the resource footprint of the API server and etcd as outlined in tbl. 2. However, this study project focuses on the scalability of the controller’s side only. Scalability limitations and implications of the control plane are out of the scope of this thesis.

resource	depends on
CPU	rate and size of API objects converted, encoded, decoded, rate and size of watch events dispatched to clients
memory	number and size of API objects stored in the watch cache
network bandwidth	rate and size of API requests and watch events
disk I/O	rate and size of serialized API objects read from and written to disk

Table 2: Resource implications on API server and etcd

3.2 Requirements for Horizontal Scalability

To scale Kubernetes controllers horizontally, the restriction of having only a single active instance at any given time needs to be lifted. For this, a concept for sharding API objects must be introduced which distributes ownership of different objects across multiple instances. Nevertheless, it must still be prevented that multiple instances own a single API object at the same time or act on it concurrently.

This section defines precise requirements for a sharding mechanism for Kubernetes controllers. The requirements themselves are inspired by the required mechanisms for sharding in distributed databases (sec. 2.7). If these requirements are met, the resources listed in tbl. 1 can be distributed across multiple instances which makes the controller horizontally scalable.

Req. 1: Membership and Failure Detection

Firstly, the system needs to populate information about the set of controller instances. In order to distribute object ownership across instances, there must be a mechanism for discovering members of the sharded setup. Additionally, this mechanism needs to provide information on the availability of individual instances. Instance failures need to be detected for remediating them and restoring the system's functionality quickly and automatically.

Because Kubernetes controllers are part of a distributed system and deployed in cloud environments (on commodity hardware), the sharding mechanism must expect that instances are restarted frequently and can fail at any time. Also, instances can be dynamically added or removed to adapt to growing or decreasing demand. Furthermore, instances will typically be replaced in quick succession during rolling updates. For these reasons, voluntary disruptions should be handled specifically and gracefully (scale-down, rolling updates) to achieve fast rebalancing.

Req. 2: Partitioning

Secondly, the sharding mechanism must include a partitioning algorithm for determining ownership of a given object based on information about the set of available controller instances. It must map every sharded API object to exactly one instance. Additionally, the partitioning algorithm needs to provide a balanced distribution even

with a small number of instances (e.g., less than 5).

As an input for the partitioning algorithm, a partition key is needed. It needs to apply to all available API resources. Hence, there should be a generic function for determining the partition key of a given API object. Furthermore, controllers commonly own and manage objects of different kinds for implementing the logic of a given API resource. E.g., the `Deployment` controller owns and manages `ReplicaSet` objects. As controllers watch the owned objects to trigger reconciliation of the owning object on relevant changes to owned objects, the partitioning algorithm must support assigning related objects to the same instance. For this, all owned objects should map to the same partition key as their owners.

Req. 3: Coordination and Object Assignment

Next, the sharding mechanism must provide some form of coordination between individual controller instances and assign API objects to the instances based on the partitioning results. As Kubernetes controllers realize the desired state of the system asynchronously, there is no direct communication of intent between the issuing user and the responsible controller. Hence, there is no need for partition-based coordination of client requests. Individual controller instances need to know which objects are assigned to them in order to perform the necessary reconciliations. However, individual instances don't need to be aware of all object assignments. The object assignment needs to be transparent and must not change any existing API semantics.

As described in tbl. 1, the resource requirements of Kubernetes controllers don't only depend on the actual reconciliations but also depend on the controllers' caches and underlying watch connections. The sharding mechanism can only make controllers horizontally scalable if the instances' caches and watch connections are filtered to only contain and transport the API objects that are assigned to the respective instances. Otherwise, the mechanism would replicate the cached data and the corresponding resource requirements, which contradicts the main goals of distributing load between instances. Another important requirement is that individual instances need to be able to retrieve the object assignment information after restarts. I.e., object assignments must be stored persistently.

Furthermore, there must not be a single point of failure or bottleneck for reconciliations. This means, the sharding mechanism must not add additional points of failure on the critical path of API requests and the reconciliations themselves, which would limit the mechanism's scalability again. During normal operations, reconciliations should not

be blocked for a longer period.

Req. 4: Preventing Concurrency

Additionally, even when object ownership is distributed across multiple controller instances, concurrent reconciliations for a single object in different instances are still not allowed. I.e., the purpose of the current leader election mechanism must not be violated. Only a single instance is allowed to perform mutations on a given object at any given time. A strictly consistent view on object assignments is not needed for this though. The only requirement is, that multiple controller instances must not perform writing actions for a single object concurrently. In other words, the sharding mechanism must not involve the replication of objects. All API objects are only assigned to a single controller instance.

Req. 5: Incremental Scale-Out

The final requirement is that the introduced sharding mechanism provides incremental scale-out properties. This means that the capacity and throughput of the system increase almost linearly with the added resources, i.e. with every added controller instance.

4 Design

This chapter presents a design for implementing sharding in Kubernetes controllers to make them horizontally scalable. The presented design includes well-known approaches for sharding in distributed databases and applies them to the problem space of Kubernetes controllers. Approaches are chosen to fulfill the requirements listed in sec. 3.2. First, the overall architecture of the design is introduced. After that, different aspects of the design are described in detail.

4.1 Architecture

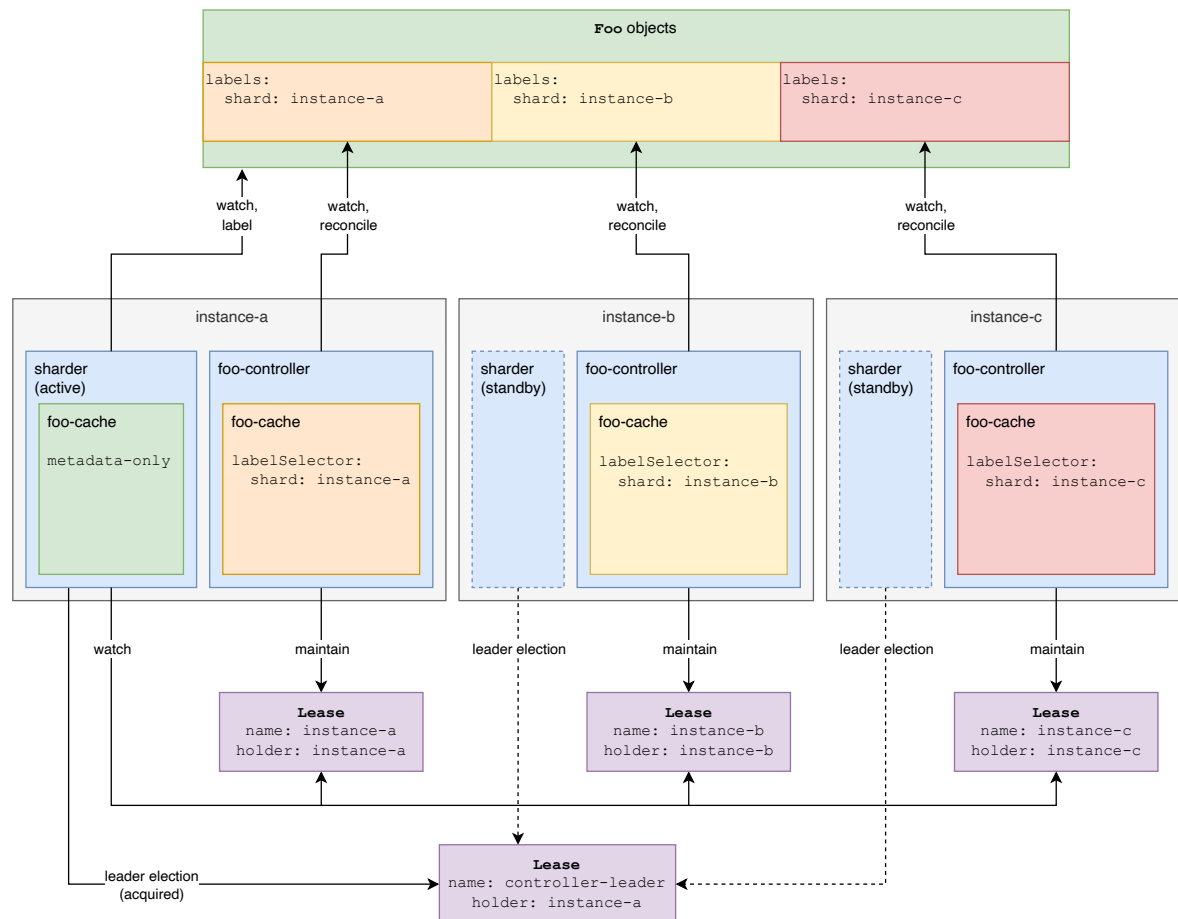


Figure 3: Kubernetes controller sharding architecture

The design heavily builds upon existing mechanisms of the Kubernetes API and controllers. It reuses established approaches from distributed databases and maps them

to Kubernetes to implement sharding in a “Kubernetes-native” manner. This is done to leverage as much of the existing controller infrastructure as possible.

Firstly, multiple instances of the controller are deployed with the same configuration. All instances are equal, though one of them is elected to be the “sharder” using the usual lease-based leader election mechanism. In addition to running the main controllers, the sharder is also responsible for performing sharding-related tasks. Most importantly, this includes assigning API objects that should be sharded to individual controller instances. It persists object assignments by setting the `shard` label on the objects themselves to the ID of the responsible instance. The sharder components themselves are responsible for all API objects and bootstrapping the sharding mechanism. Hence, the sharder components are only executed as long as the instance is the current leader as they must not run in multiple instances concurrently. I.e., there is only a single active sharder at any given time and the sharder components in additional instances are on standby. The sharder uses a metadata-only watch for all the sharded objects to minimize the resulting resource footprint.

Each controller instance (“shard”) maintains an individual lease (“shard lease”), which is used as a heartbeat resource to inform the sharder about the health status of all instances. The sharder watches the shard leases in addition to the sharded API objects themselves in order to react to state changes, e.g., the addition or removal of shards. The main controllers in all shards watch and cache only the objects they are responsible for by filtering the watch connection using a label selector on the `shard` label.

4.2 Membership and Failure Detection

To address requirement 1, a lease-based membership and failure detection mechanism are used. The mechanism is heavily inspired by Bigtable but adapted to use Kubernetes `Lease` objects stored via the API server instead of leases stored in Chubby [18].

The sharder performs the usual leader election to ensure that there is only one active sharder at any given time. Additionally, each shard maintains an individual shard lease. Like usual leader election, the `Lease` API resource with the same fields and semantics are used for this purpose. However, shard leases use individual names that are equal to the shard’s instance ID. The instance ID needs to be stable between restarts of the shard, so the hostname is used, i.e., the controller’s `Pod` name. On startup, the shards acquire the respective shard lease using the instance ID as name and holder identity. As with the usual leader election, the shards periodically renew their leases

after a configured duration. As long as the shard actively holds its lease, it watches and reconciles objects that are assigned to itself. If the shard fails to renew its lease within a configured period, it needs to stop all sharded controllers immediately and terminate.

The sharder controller watches all shard leases, which allows it to discover all available instances and immediately react to state changes. As long as a shard holds its lease, the sharder considers it for partitioning. On shard termination, the shard unsets the lease's holder identity field to release its lease. This signals a voluntary disruption to the sharder, which immediately removes the instance from the partitioning. With this, all objects assigned to the terminated shard are moved to other available instances.

Shard failures are detected by checking the leases' `renewTimestamp` field. If a shard fails to renew its lease within `leaseDurationSeconds`, the lease is considered expired. To decrease the sensitivity of the system, the sharder waits for another `leaseDurationSeconds` and allows the instance to renew its lease again. If the shard comes available again by renewing its lease it is considered ready again. If the shard fails to renew its lease though, the sharder considers the instance unavailable. However, before it removes the instance from partitioning, it tries to acquire the shard lease once. This is done to ensure the API server is functioning and the sharder doesn't act on stale lease data (similar to Bigtable [18]). If the sharder is able to acquire the shard lease, it considers the instance dead, removes it from partitioning, and moves objects to available instances. Afterward, the sharder doesn't touch the shard lease anymore. With this, the shard is allowed to acquire its lease once it comes up again after the lease duration set by the sharder has passed. If this is successful, the instance is considered ready and included in partitioning again.

The sharder garbage collects leases of dead shards after a period of inactivity. With this, leases of voluntarily terminated instances as well as failed instances get cleaned up eventually.

4.3 Partitioning

To address requirement 2, a variant of consistent hashing is used [22], [23]. With this, partitioning is done similarly as in Apache Cassandra [16] and Amazon Dynamo [24] but adapted to use Kubernetes API object metadata as input. Consistent hashing is chosen because it minimizes movement on the addition and removal of instances. Also, it provides a simple and deterministic algorithm for determining the responsible shard

solely based on the set of available instances. Therefore, no state of the partitioning algorithm must be stored apart from the instance states available through the membership mechanism (sec. 4.2). The sharder can simply reconstruct the hash ring based on this information after a restart or leader transition without risking inconsistency or unstable assignments. In order to provide a balanced distribution even with a small number of instances, every instance ID is mapped to a preconfigured number of tokens on the hash ring (virtual nodes) [23].

For determining ownership of a given API object, a partition key is derived from the object's metadata. By default, it consists of the object's API group, kind, namespace, and name. This partition key is chosen in order to equally distribute objects of different kinds with the same namespace and name across shards. Additionally, the object's UID is added to distinguish between different instances of an API object with the same namespace and name. As all API objects share these common metadata fields, this approach can be applied to all API resources likewise. Also, it allows the sharder to use lightweight metadata-only watches. To assign owned objects to the same shard as their owners, the owner's partition key can be determined from the owner reference in the owned object's metadata.

4.4 Coordination and Object Assignment

Realizing coordination and object assignment (requirement 3) in Kubernetes controllers is very simple in comparison to distributed databases. In contrast to sharding in databases, no request coordination is needed because clients don't communicate directly with the controller instances. Because of this, object assignments don't need to be persisted in a dedicated store or storage section like in Bigtable, MongoDB, or Spanner [18], [20], [21]. Furthermore, instances don't need to be aware of assignment information of objects they are not responsible for. Hence, there is no need to propagate this information throughout the system as it is done in many distributed databases [16], [24].

In the presented design, object assignment is done by the sharder controller by labeling API objects with the instance ID of the responsible shard. For each object kind that should be sharded, one sharder controller is started that uses the mechanisms described above for discovering available instances and determining assignments. Persisting assignments in the API objects themselves are done to make use of existing controller infrastructure for coordination. By labeling the objects themselves, controllers can simply use a filtered watch to retrieve all API objects that are assigned to them. For

this, a label selector on the `shard` label for the shards' instance ID is added to the controllers' watches. With this, the controllers' caches and reconciliations are already restricted to the relevant objects and no further coordination between controllers is needed. Additionally, controllers already rebuild watch connections on failures or after restarts automatically. This means object assignment information is automatically rebuilt on failures or restarts without any additional implementation. Using a filter watch and cache additionally addresses requirement 5, as all relevant resource requirements (tbl. 1) are distributed across multiple instances. The system's capacity roughly increases linearly with each added instance.

The sharder itself is not on the critical path for all reconciliations. As soon as objects are assigned to an instance, the sharder doesn't need to be available for reconciliations to happen successfully. Reconciliations of new objects might however be delayed by a short period on leader transitions. As there are multiple instances of the sharder controller on standby, handover generally happens quickly. Also, handover can be sped up by releasing the leader election lease on voluntary step-down.

4.5 Preventing Concurrency

With the presented design, concurrent writes from different instances to the same API object are already prevented as long as object assignments don't change. To fulfill requirement 4, concurrent mutating reconciliations must additionally be prevented when moving objects between shards. In this case, the sharder must ensure that the old instance has stopped working on the given object before another instance picks it up.

The first case that needs to be considered is when moving objects from a ready shard to another shard. This can be required for rebalancing during scale-out or a rolling update, i.e. when a new instance is added. As the controller system is asynchronous and pull-/watch-based, the sharder cannot immediately reassign objects. The watch connection and cache of the old instance might receive the reassignment event later than the new instance, which could lead to concurrent and conflicting reconciliations. In other words, the sharder can't know if and when an instance has stopped working on an object that is supposed to be moved. To address this challenge, the old instance needs to acknowledge the reassignment and confirm it has stopped working on the object. When the sharder needs to move an object from a ready shard to another, it adds a `drain` label indicating that the object should be moved. As soon as the old shard observes the `drain` label it must remove it from the object as well as the `shard` label, marking the object as unassigned. When the sharder observes that the object is

unassigned it reassigns it to the desired instance as it does for new objects. Following this protocol, concurrent reconciliations by the old and the new shard are prevented. Also, it facilitates fast rebalancing with ready and responsive instances.

A second case to consider is when moving objects from a terminating instance to a ready instance. This can be required for rebalancing during scale-in or a rolling update, i.e. when an existing instance is removed. In this case, the shard releases its shard lease to signal voluntary termination to the sharder. As soon as the sharder observes this change to the shard lease it immediately reassigns API objects to another instance. A `drain` operation is not performed in this case because the instance has already acknowledged that it stopped working on the objects by releasing its shard lease.

The last case that needs to be considered is when moving objects from a dead instance, i.e. for rebalancing after an instance failure. Once the sharder has detected a shard failure and acquired the shard lease as described in sec. 4.2, objects are reassigned immediately without performing a `drain` operation. If the shard eventually becomes ready again, objects can be assigned to it again once it acquires its lease just like when a new instance is added. A special case occurs when an instance failure is detected after initiating a `drain` operation. In this case, the sharder removes the `drain` label by itself when updating the `shard` label to prevent performing the `drain` operation with the shard that the object is supposed to be assigned to.

5 Implementation

This chapter describes how the design presented in chapter 4 is implemented in practice. First, the webhosting operator is introduced which serves as an example operator for implementing sharding in Kubernetes controllers. Then, the overall architecture of the implementation is described, followed by detailed descriptions of the most important aspects. The presented implementation is used for evaluation in chapter 6.

5.1 Webhosting Operator

To implement and evaluate the presented design, an operator is needed that serves as an example. The sample operator should fulfill the following criteria so that the most important aspects of sharding in Kubernetes controllers can be covered and evaluated:

1. The operator should be composed of a single controller performing reconciliations for one custom resource. With this, the operator's capacity and throughput can be measured and compared more easily as they are mainly determined by the number of objects and event rate for this resource.
2. In addition to watching the reconciled objects, the controller should also watch all other relevant objects. This includes owned objects, that the controller manages for realizing the intent of the reconciled objects. Also, it should watch objects that might be referenced by multiple reconciled objects. Both of these are common patterns in controllers, which makes the example operator a good representative.
3. Finally, the operator needs to follow controller best practices [13]. Most importantly, it should be level-based, meaning it performs reconciliations purely based on the desired and currently observed state but independent of observed changes to these states. However, it should be edge-triggered for performance and responsiveness, i.e. start reconciliations based on relevant change events. Reconciliations should be short and must not wait for the desired and actual state to converge. Lastly, the operator should hold all relevant state in memory, meaning it should cache all API resources it works with. Following these best practices in the example operator is important because many controllers are implemented in compliance with them. Hence, the evaluation can only provide meaningful insights if the measured implementation follows the best practices.

Based on these criteria, the webhosting operator¹ was designed and built. The operator and its resources are modeled to form a webhosting-like platform, where customers can create and manage websites in a declarative manner via the Kubernetes API. Websites reside in a project namespace, have a name, and specify a website theme, which defines color and font family. The desired state of websites is declared via Kubernetes resources and the operator manages the required web servers and exposes them to the internet.

```
apiVersion: webhosting.timebertt.dev/v1alpha1
kind: Theme
metadata:
  name: exciting
spec:
  color: darkcyan
  fontFamily: Menlo
---
apiVersion: webhosting.timebertt.dev/v1alpha1
kind: Website
metadata:
  name: homepage
  namespace: project-foo
spec:
  theme: exciting
```

Code 5: Example webhosting operator objects

Three API resources are involved when managing websites using the webhosting operator:

1. **Namespaces** are used to separate the websites of different customers and projects. I.e., one could provide new customers with access to a dedicated namespace upon registration to the service.
2. **Theme** is a custom resource of the webhosting operator and part of the `webhosting.timebertt.dev` API group. **Themes** are cluster-scoped and can be referenced by **Websites** in all namespaces. They are managed by the service administrator and configure an HTML color and font family.
3. **Websites** are also part of the operator's custom API group, but are namespaced. A **Website** object represents a single website managed by a customer in a respective project namespace. It references exactly one of the offered **Themes**. For each **Website** object, the operator creates and configures an

¹<https://github.com/timebertt/kubernetes-controller-sharding/tree/v1.0>

nginx `Deployment` to host a simple static website that displays the website's name and technical ID ("server name"). The website uses the color of the referenced `Theme` as a background color as well as the declared font family. Additionally, the `Deployment` is exposed via an `Ingress` object on the URL path `/<namespace>/<website-name>`.

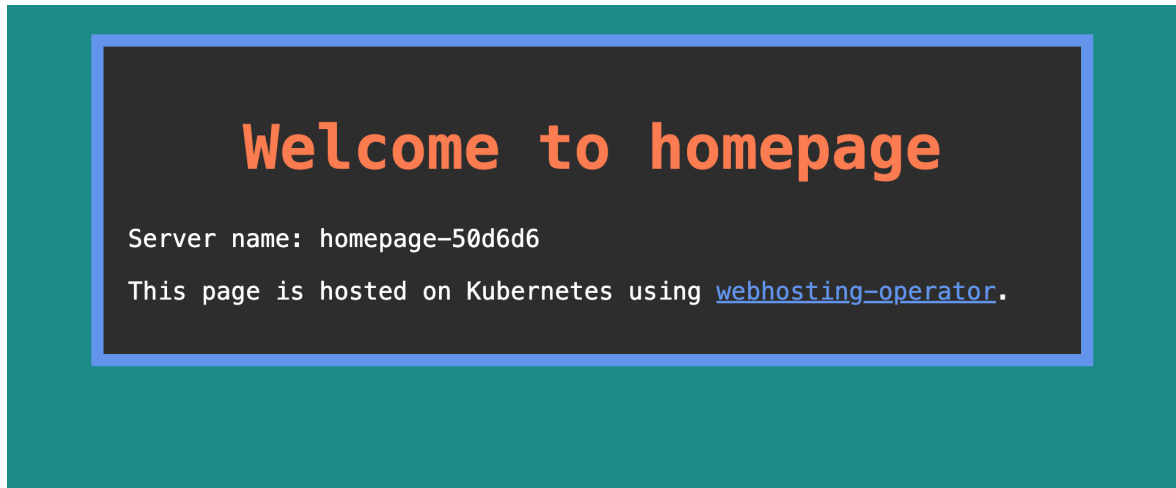


Figure 4: Sample website managed by webhosting operator

The webhosting operator contains a single controller for realizing the webhosting functionality. On each reconciliation of a `Website` object, it ensures that owned objects of the `Website` are correctly configured: the nginx `ConfigMap` and `Deployment` as well as `Service` and `Ingress` for exposing the `Deployment`. It adds owner references from all of these objects to the owning `Website` object. In addition to watching `Website` objects, it also watches these owned objects and enqueues the owning `Website` for reconciliation on relevant changes, e.g. when the `Deployment` object's readiness status changes. This facilitates self-healing capabilities of websites, as the operator ensures that all required objects have the desired state. Additionally, it allows the controller to immediately report the status of the `Website` object in the `status.phase` field based on the status of the nginx `Deployment`.

Furthermore, the controller watches all available `Themes` and enqueues all `Website` objects that reference a `Theme` for reconciliation, if its specification changes. With this, changes to `Themes` are immediately rolled out to all `Websites` that use them. Additionally, `Website` reconciliations can be triggered by annotation changes for load test purposes. The reconciliations themselves are very short, making the controller responsive and scalable.

5.2 Architecture

The webhosting operator is based on the controller-runtime library. The library provides reusable constructs for building custom Kubernetes controllers and operators in the go programming language. In controller-runtime, the **Manager** is the most important construct. A **Manager** is initialized and started once per operator as the root component and manages all other components like controllers, webhook server and handlers, leader election, metrics endpoints, loggers, etc. [25], [26]

Individual controllers can be implemented by creating a corresponding **Controller** construct and configuring a **Reconciler**, watches, and event handlers. The **Reconciler** contains the business logic of a controller. It is invoked with a reconciliation request containing an object's name and namespace. On each invocation, it has to compare the actual state of the system with the desired state of the given object and perform actions to converge both, i.e. perform reconciliation of the object. The configured watches and event handlers are responsible for enqueueing reconciliation requests in response to watch events. Typically, **Predicates** are used to filter for relevant watch events emitted by the API server. The **Controller** itself ensures that all components including cache, event handlers, work queue, and worker routines are correctly set up. All **Controllers** are registered with the **Manager** which then ensures that the controllers are started when the leader election is won and stopped as soon as the leader election is lost. It also injects shared dependencies like client, cache and loggers into the individual controllers. [26]

As part of this thesis, the presented design (chapter 4) was fully implemented generically in the controller-runtime library that allows reusing the mechanisms in all operators based on the library. The webhosting operator makes use of the implemented sharding mechanisms in controller-runtime for demonstration and evaluation purposes.

For adding controller sharding to an operator based on controller-runtime, there are two places involved: when configuring the **Manager** and when setting up a sharded **Controller**. First, sharding has to be enabled by setting `manager.Options.Sharded=true`. With this, the **Manager** is configured to maintain a shard lease while it is running (sec. 5.3). Furthermore, it adds a cache to the **Manager** which uses a label selector `shard=<shard-id>` for all objects. This cache is then used by sharded controllers as a filtered cache that only contains objects that the shard is responsible for, hence it is referred to as the sharded cache. In controller-runtime, the cache is empty on startup and watches are started on-demand when a controller reads from the cache. The **Manager**'s default cache (unfiltered) is

still available for retrieval of non-sharded objects. However, as long as controllers use the correct cache when reading objects, watches will only be started in one cache for any `GroupVersionKind` and thus are filtered correctly. For development purposes, the shard ID can optionally be overwritten. Also, the `ShardMode` option allows configuring whether the manager should only run the sharder components (under leader election), only the shard components, or both (default).

```
mgr, err := manager.New(restConfig, manager.Options{
    // enable sharding for this manager
    Sharded: true,
    // optionally configure overwrites for development purposes
    ShardID: "custom-shard-id",
    ShardMode: sharding.ModeSharder,
})
```

Code 6: Configuring the controller-runtime manager for sharding

There are two sharder components: the lease controller and the sharder controllers. Both are added in all instances, but they are only running when the instance has acquired leadership. The lease controller (sec. 5.4) is running only once per leader and is mainly responsible for acquiring leases of unavailable shards (sec. 4.2). The sharder controllers are coupled to the setup of the sharded controllers themselves, though they only run in the leader.

```
controller, err := builder.ControllerManagedBy(mgr).
    For(&webhostingv1alpha1.Website{}, builder.Sharded{}).
    // watch deployments in order to update phase on relevant changes
    Owns(&appsv1.Deployment{}, builder.Sharded{}).
    // ...
    // watch themes to roll out theme changes to all referencing websites
    Watches(
        &source.Kind{Type: &webhostingv1alpha1.Theme{}},
        handler.EnqueueRequestsFromMapFunc(r.MapThemeToWebsites),
        builder.WithPredicates(predicate.GenerationChangedPredicate{}),
    ).
    Build(reconciler)
```

Code 7: Setup of a sharded controller

For setting up a sharded controller, the `builder` package must be used. With this, sharded controllers are configured like normal controllers however with an additional `builder.Sharded` option. This option is supplied to the controller's main object

type (in `For`) and all additional object types that should be sharded (in `Owns` and `Watches`). When setting the `Sharded` option, the builder performs the following tasks:

- it adds a sharder controller (sec. 5.5) for the corresponding `GroupVersionKind`
- it modifies the main object controller (sec. 5.6) to properly handle the sharding mechanisms

The next sections explain the implementation of all components in detail.

5.3 Shard Lease

When configured for sharding, the manager keeps performing leader election (sec. 2.6) and runs the sharder components only when it has successfully acquired leadership. In addition to that, the manager also maintains an individual shard lease (sec. 4.2). The mechanism executes the same leader election code, however with an individual lease name. For shard leases, the instance's hostname is used for both the lease name and holder identity. As long as the manager is able to renew the shard lease it keeps running the sharded controllers.

When adding controllers and other `Runnables` to a manager via `manager.Add` they can signal whether they need to run under leader election or not by implementing the `LeaderElectionRunnable` interface. For example, controllers implement this interface by default to always run with leader election. A new interface is introduced to distinguish between usual and sharded controllers: `ShardedRunnable`.

```
// ShardedRunnable knows if a Runnable needs to be run in the sharded mode.
type ShardedRunnable interface {
    // IsSharded returns true if the Runnable needs to be run in the sharded
    ↪ mode.
    IsSharded() bool
}
```

Code 8: The `ShardedRunnable` interface

When creating a sharded controller using the builder, the manager recognizes the `ShardedRunnable` implementation and starts it as soon as the shard lease has been acquired even if the instance is currently not the leader. Similar to the usual leader election, the manager process exits immediately when it fails to renew its shard lease. When

the manager is stopped, it engages a graceful termination procedure and releases its shard lease to signal voluntary disruption, i.e. it sets `spec.holderIdentity=null`.

5.4 Lease Controller

The lease controller watches the shard leases that individual instances maintain. For a given shard lease, it first determines the instance's state and then adds the state to the lease object as the `state` label for observability purposes. The shard states are defined by the following rules:

- **Ready**: the lease is held by the shard itself (`metadata.name` is equal to the `holderIdentity`) and has not expired (`renewTime + leaseDurationSeconds` has not passed yet)
- **Expired**: the lease is held by the shard but has expired up to `leaseDurationSeconds` ago
- **Uncertain**: the lease is held by the shard and has expired more than `leaseDurationSeconds` ago
- **Dead**: the lease is not held by the shard anymore, either because the shard released its lease or it was acquired by the sharder (lease controller)
- **Orphaned**: the lease is in state **Dead** and has expired at least 1 minute ago

The main responsibility of the lease controller is to act on leases of unavailable shards. Once shard leases transition to state **Uncertain**, it acquires the lease by setting `holderIdentity` to `sharder`. For this, it uses a `leaseDuration` twice as long as the `leaseDuration` of the shard. Acquiring the shard lease is done to ensure the API server is functioning before removing the shard from partitioning (sec. 4.2). Additionally, the lease controller deletes shard leases in the state **Orphaned** to not pollute the system. The shard controller reconciles leases as soon as their state changes, either because of a create/update event or because one of the relevant durations has passed.

5.5 Sharder Controllers

For every sharded object kind, the controller builder adds a sharder controller that runs under leader election. It is responsible for assigning the sharded objects to available shards by setting the `shard` label based on the shard state information. For this, it starts a lightweight metadata-only watch/cache for the respective object kind. As soon

as objects are created or need to be re-assigned the controller performs reconciliation. Additionally, it watches the shard leases and enqueues relevant sharded objects when a shard's availability changes.

On every reconciliation, the controller lists all shard leases from the cache. It then constructs a hash ring including all shards that are not in the state `Dead` or `Orphaned`. As this operation is executed for every sharder reconciliation, it is saved for reuse in a dedicated cache that is shared across all sharder controllers belonging to a given sharded controller. The cached ring is recalculated when a lease is updated. In the hash ring, 100 tokens are added per shard by default for better distribution of objects across a low number of shards (inspired by groupcache [27]).

```
type Hash func(data []byte) uint64

type Ring struct {
    hash      Hash // defaults to 64-bit implementation of xxHash (XXH64)
    tokensPerNode int // defaults to 100

    tokens      []uint64
    tokenToNode map[uint64]string
}

func (r *Ring) AddNodes(nodes ...string) {
    for _, node := range nodes {
        for i := 0; i < r.tokensPerNode; i++ {
            t := r.hash([]byte(fmt.Sprintf("%s-%d", node, i)))
            r.tokens = append(r.tokens, t)
            r.tokenToNode[t] = node
        }
    }

    // sort all tokens on the ring for binary searches
    sort.Slice(r.tokens, func(i, j int) bool {
        return r.tokens[i] < r.tokens[j]
    })
}
```

Code 9: Adding shards to the hash ring

After calculating the hash ring, the object is mapped to a partition key. For the main object kind of sharded controllers, the partition key follows this pattern:

```
<kind>.<group>/<namespace>/<name>/<uid>
```

For any owned objects of the main object kind, the partition key of the owner is

calculated using the object's owner references. The calculated partition key is then used to determine the desired shard for the object by hashing the key onto the hash ring.

```
func (r *Ring) Hash(key string) string {
    // Hash key and walk the ring until we find the next virtual node
    h := r.hash([]byte(key))

    // binary search
    i := sort.Search(len(r.tokens), func(i int) bool {
        return r.tokens[i] ≥ h
    })

    // walked the whole ring
    if i == len(r.tokens) {
        i = 0
    }

    return r.tokenToNode[r.tokens[i]]
}
```

Code 10: Consistent hashing of object keys

If the object is not assigned yet, the sharder controller simply patches the object's **shard** label to the desired shard. However, if the object is already assigned to a different shard, the sharder controller first adds the **drain** label to the object to wait for acknowledgment by the current shard (sec. 4.5). This operation is only performed for the main object kind of the sharded controller, as it does not perform mutating actions on owned objects as long as it isn't responsible for the owning object. Hence, concurrency is already prevented by the drain operation of the main object and doesn't need to be handled separately for owned objects.

5.6 Object Controller

The actual sharded controller is modified by the builder to handle the sharding mechanisms. First, the controller implements the **ShardedRunnable** interface so that all instances run the controller and not only the leader. Also, the builder constructs a cache that automatically delegates all read operations to either the sharded (filtered) cache or unfiltered cache according to whether they are configured to be sharded or not (as shown in code 7). This cache is injected into the object controller instead of

the unfiltered cache. With this, the sharded controller automatically uses the filtered cache as desired while it is still able to read other non-sharded objects.

Furthermore, the builder makes sure that the controller respects the shard assignments and acknowledges drain operations. This is done by wrapping the actual reconciler with another generic reconciler. On each reconciliation, it checks if the object is still assigned to the respective shard by reading the `shard` label. If the object is not assigned to the shard anymore, it is discarded. Before delegating reconciliation requests to the actual reconciler, it also checks the `drain` label. In case the `drain` label is present, the controller removes both the `shard` and `drain` labels from the object and stops reconciling it. Lastly, the controller's predicates are adjusted to immediately react to events in which the object has the `drain` label. This effectively allows setting up a sharded controller without changes to the controller's code.

```
func (r *Reconciler) Reconcile(ctx context.Context, request reconcile.Request)
↪ (reconcile.Result, error) {
    // read object from cache...
    labels := obj.GetLabels()
    if shard, ok := labels[sharding.ShardLabel]; !ok || shard != r.ShardID {
        // forget object
        return reconcile.Result{}, nil
    }

    if _, drain := labels[sharding.DrainLabel]; drain {
        // acknowledge drain operation
        patch := client.MergeFromWithOptions(obj.DeepCopyObject().(client.Object),
        ↪ client.MergeFromWithOptimisticLock{})
        delete(labels, sharding.ShardLabel)
        delete(labels, sharding.DrainLabel)

        if err := r.client.Patch(ctx, obj, patch); err != nil {
            return reconcile.Result{}, fmt.Errorf("error draining object: %w", err)
        }

        // forget object
        return reconcile.Result{}, nil
    }

    // we are responsible, reconcile object
    return r.Do.Reconcile(ctx, request)
}
```

Code 11: Wrapping reconciler for sharded controllers (simplified)

6 Evaluation

This chapter describes how the presented implementation (chapter 5) is used for the evaluation of the proposed design (chapter 4). First, the deployment environment is described. Next, the monitoring setup is introduced which is used to observe the webhosting operator and to take measurements. After that, the conducted experiment is described in detail. Last, the experiment's results are presented and discussed.

6.1 Deployment

For evaluation of the proposed design, the webhosting operator implementation is deployed to a Kubernetes cluster along with other tools for monitoring and experimentation. The Kubernetes cluster¹ is managed by Gardener² and runs on Google Cloud Platform (region `eu-west-1`). The cluster runs Kubernetes version `1.24`.

For isolation purposes and accurate measurements during load tests, the following worker pools are used to run the different components:

worker pool	instance type	components	count
system	n1-standard-8	cluster system, monitoring, ingress	1
operator	n1-standard-4	webhosting operator	1
websites	n1-standard-8	website deployments	10

Table 3: Worker pools of the evaluation cluster

Both the `operator` and `websites` worker pools are configured with taints [4] to repel all workload pods by default. With this, all pods are scheduled to the system worker pool if not configured otherwise. Kyverno policies [28] are used to add tolerations and scheduling constraints to all webhosting operator and website pods so that they are scheduled to the respective dedicated worker pools. To reduce the resource requirements of load tests, the websites worker pool is configured to run up to 800 pods per node. Additional policies are used to replace the `nginx` image in website pods with the `pause` image during load tests. With this, websites are not functional but have a low memory footprint without affecting the experiment's measurements.

¹Cluster specification

²<https://github.com/gardener/gardener>

6.2 Monitoring

For observing and measuring the implementation, a monitoring setup is deployed in the Kubernetes cluster in addition to the webhosting operator itself. The monitoring setup serves two purposes: for general evaluation that the sharded controllers are working as desired and for providing accurate measurements of the operator’s resource usage.

The deployed monitoring setup is based on kube-prometheus¹, which is a collection of monitoring tools and configurations for Kubernetes clusters. Most importantly, it includes a Prometheus [29] instance which collects metrics for containers running on the cluster and metrics about the state of Kubernetes API objects. Container metrics are scraped from the kubelet’s cadvisor endpoint [4], kube-state-metrics² is used as an exporter for metrics about API objects. Furthermore, the setup includes a Grafana [30] instance, which visualizes the metrics collected by prometheus in different dashboards.

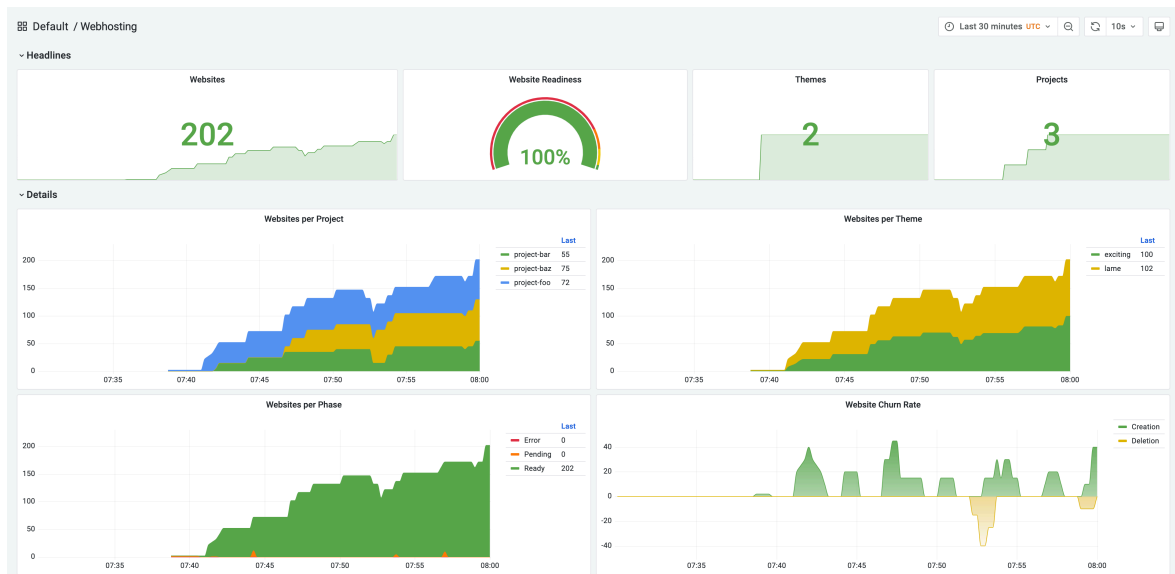


Figure 5: Webhosting dashboard

In addition to the components included in kube-prometheus, the webhosting exporter³ is deployed which exposes metrics about the state of websites. It is implemented using the kube-state-metrics library and complements the metrics offered by kube-state-metrics by adding similar metrics about the webhosting operator’s API objects. For example, one metric indicates the websites’ phase, and another one states which shard

¹<https://github.com/prometheus-operator/kube-prometheus>

²<https://github.com/kubernetes/kube-state-metrics>

³webhosting exporter source code

websites are assigned to. Additionally, the webhosting exporter is used to collect metrics about the state of the operator's shards, which is determined from the individual shard leases.

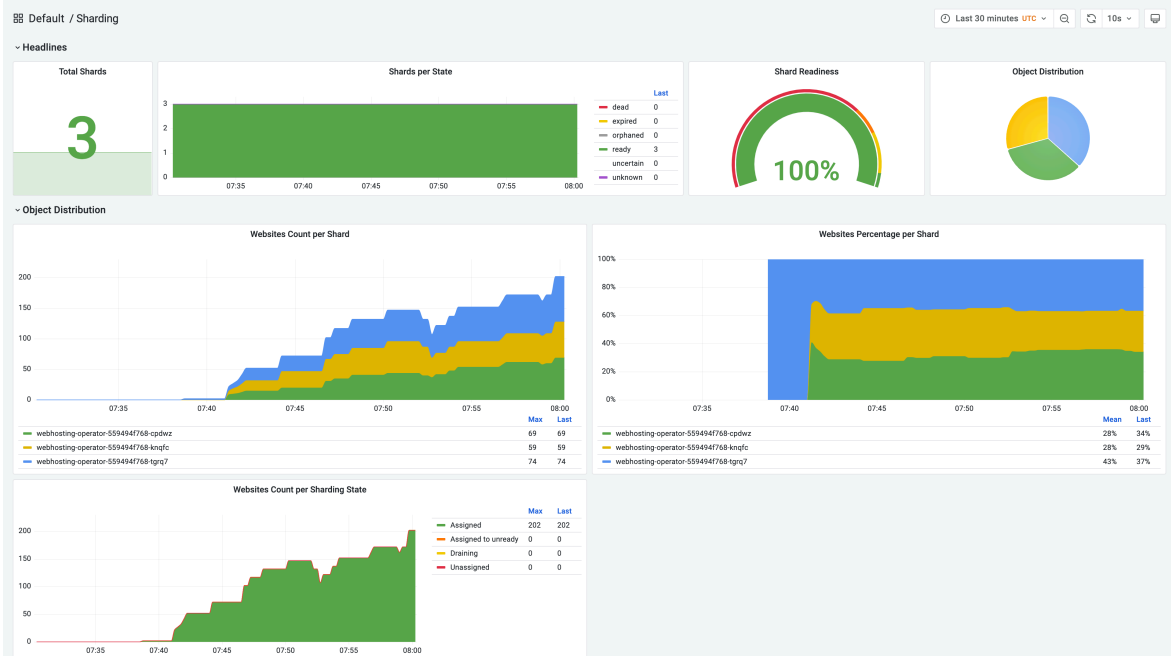


Figure 6: Sharding dashboard

Finally, Prometheus is configured to collect metrics from the webhosting operator as well. It exposes metrics from the controller-runtime library about controllers, work queues, and the Kubernetes client (fig. 7). For example, these metrics offer information on the rate and duration of reconciliations, and for how long objects are queued before being reconciled. Additionally, metrics are added about the sharder controllers' actions, i.e. assignment and drain operations.

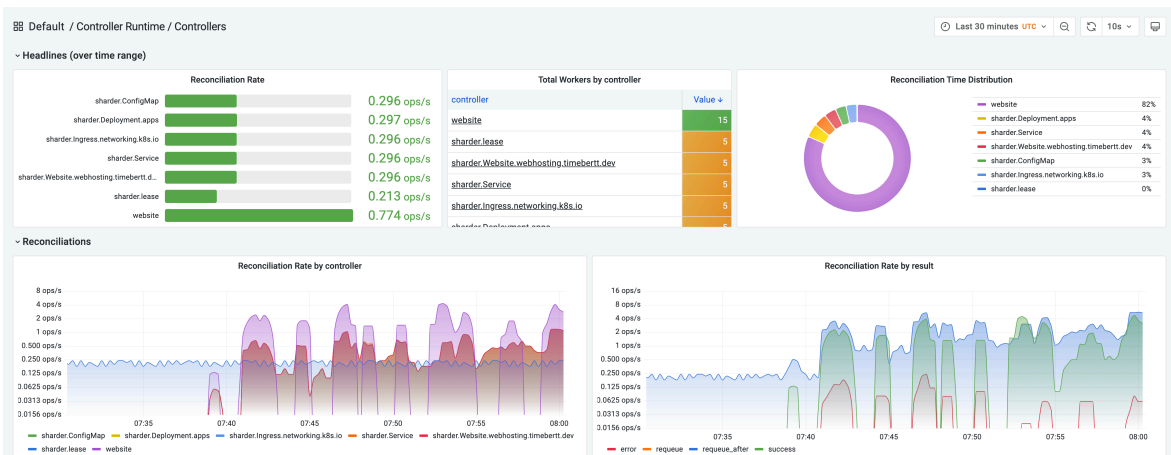


Figure 7: Controller-runtime dashboard

For general evaluation of the implementation, a sample generator¹ is implemented to create a configurable number of random website objects in different namespaces. Fig. 5 shows the overview dashboard over webhosting API objects and their state based on metrics exposed by webhosting exporter and kube-state-metrics. It demonstrates, that all websites are successfully reconciled by the sharded operator and reach the **Ready** phase as expected. The sharding overview in fig. 6 indicates that all 3 shards are healthy and properly maintain their shard lease. It also shows that responsibility for website objects is well-distributed across all 3 shards, with each shard being assigned roughly a third of all objects.

```
queries:
- name: cpu
  query:
    ↪ sum(rate(container_cpu_usage_seconds_total{namespace="webhosting-system",
    ↪ container="manager"}[2m])) by (pod)
- name: memory
  query: sum(container_memory_rss{namespace="webhosting-system",
    ↪ container="manager"}) by (pod)
- name: network_receive
  query:
    ↪ sum(irate(container_network_receive_bytes_total{namespace="webhosting-
    ↪ system", pod=~"webhosting-operator-.*"}[2m])) by
    ↪ (pod)
- name: network_transmit
  query:
    ↪ sum(irate(container_network_transmit_bytes_total{namespace="webhosting-
    ↪ system", pod=~"webhosting-operator-.*"}[2m])) by
    ↪ (pod)
```

Code 12: Queries configuration for experiments

Finally, the `measure`² tool is implemented for retrieving the relevant measurements from the Prometheus metrics store via the HTTP API [29]. It fetches time series over a given time range and stores the result matrices in CSV-formatted files for further analysis and visualization. Code 12 shows the query configuration that is used to determine the operator's resource usage in the following experiment.

¹[samples generator source code](#)

²[measure source code](#)

6.3 Experiment

Using the described deployment and monitoring setup, an experiment is conducted to evaluate the sharding implementation in the webhosting operator. The experiment aims to determine whether the relevant resources from tbl. 1 are well-distributed across the individual instances. In other words, the experiment verifies if the system's capacity and throughput are increased with the resources used by the additional instances. It assesses how well the sharded operator scales horizontally.

For this purpose, the experiment tool¹ is implemented to conduct load tests for the webhosting operator. The tool generates load by constantly creating objects and triggering reconciliations over a certain time span. It thereby increases the capacity and throughput of the system to measure the resulting resource usage. As described in sec. 6.1, the webhosting operator is deployed on nodes isolated from all other workloads. This ensures that the operator is able to consume as many resources as it needs to handle the actual capacity and throughput of the system. By this, the experiment allows concluding how much resources are needed to handle a certain capacity and throughput, and how both increase with the number of resources added to the system.

For now, only one experiment scenario is implemented, which is called **base**². In this scenario, the tool generates 50 random **Theme** objects and 20 random project namespaces. Over 10 minutes it then generates random **Website** objects in different project namespaces using different **Themes**. To make the load tests more realistic, **Websites** are also randomly deleted, though at a lower rate than creation. During the load test, the tool generates 14 random **Websites** per second and deletes 1 object per second, hence there are about 7800 **Websites** after 10 minutes. To simulate frequent changes to **Website** objects by users, the tool triggers individual reconciliations of each object once per minute but 100 reconciliations per second at maximum. Additionally, the tool also simulates occasional changes to **Theme** objects by service administrators. For this, it picks one random **Theme** per minute and mutates its specification. As described in sec. 5.1, this update triggers reconciliations for all **Website** objects referencing the **Theme**. After 10 minutes, the tool generates about 103 reconciliations per second on average with bursts of roughly 256 reconciliations per second. This load is sustained for 5 more minutes before all objects generated by the experiment are cleaned up. Fig. 8 shows how the amount of websites is increased over the experiment's timespan and how they are distributed across the operator's shards.

¹[experiment source code](#)

²[base scenario source code](#)

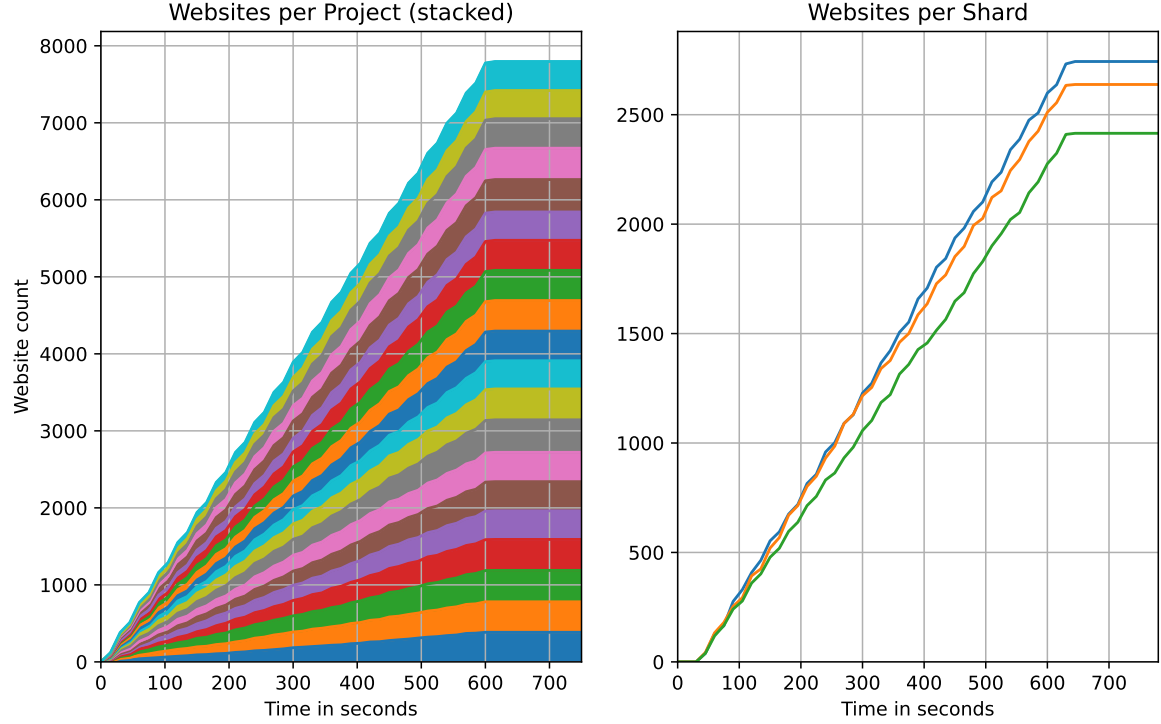


Figure 8: Load generated by base scenario

Tbl. 4 shows which metrics are used for measurements of the operator’s resource usage during the experiment. All the metrics originate from kubelet’s cadvisor endpoint. For memory measurements, the resident set size (RSS) metric is used instead of the working set size (WSS) [31]. This is because the WSS includes pages that are already freed by the process but not reclaimed by the kernel yet. The RSS on the other hand does not include such pages and hence provides a more precise measurement of how much memory the operator process actually requires.

resource	metric
CPU	container_cpu_usage_seconds_total
memory	container_memory_rss
network bandwidth received	container_network_receive_bytes_total
network bandwidth transmitted	container_network_transmit_bytes_total

Table 4: Experiment resource usage measurements

Network bandwidth measurements are split by direction: one measurement for received bytes and one for transmitted bytes. These metrics include network transfer related to Prometheus scraping the pods’ metrics endpoints. However, this can be neglected in this experiment as the transfer only occurs every 10 seconds and includes a short plain

text in comparison to the size of the transferred API objects. Note that these resource measurements only cover the operator processes but not the Kubernetes API server. Investigating the resource implications of sharding on the control plane components is out of the scope of this thesis.

The load test is executed twice: once for the singleton operator and once for the sharded operator. For this, the deployment mechanism offers a switch for deploying the operator in either mode. In the sharded mode, the operator is deployed with three instances, and in the singleton mode only with one instance. Note that singleton operators are typically deployed with multiple instances for fast failovers (active-passive HA setup) in production environments. As the passive instances of the operator also have a given base resource usage, the total resource usage of the system would be higher than in the conducted experiments. After each load test, the measurements are retrieved from Prometheus via the measure tool. Then, the measured resource usage of both tests is compared.

6.4 Results

This section compares the resource usage by individual pods in both tests for each measured resource (tbl. 4).



Figure 9: CPU usage by pod

First, fig. 9 compares the CPU usage of pods in both setups. In the sharded setup, the

pod with the highest CPU usage is the active sharder. The comparison shows that the sharder’s CPU usage is almost as high as the singleton’s usage. It only consumed 12% less CPU time than the singleton over the entire test duration. The other two shards’ total CPU time is about 48% of the singleton’s usage.

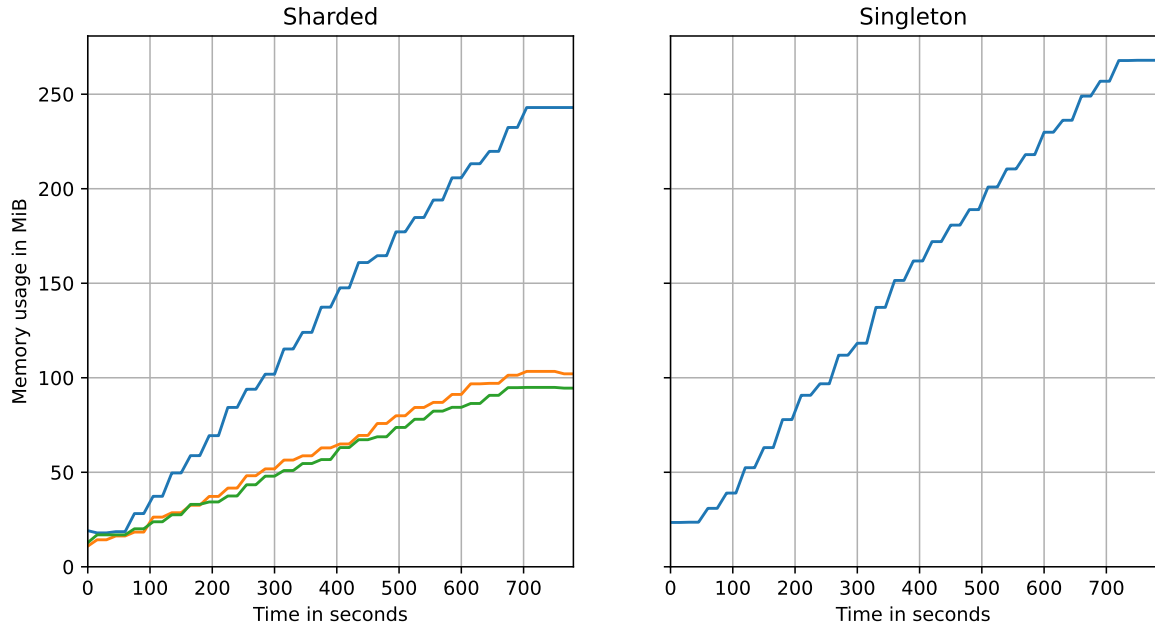


Figure 10: Memory usage by pod

The comparison of the pods’ memory usage (fig. 10) reveals that the sharder’s memory usage is almost as high as the singleton’s memory usage – similar to the CPU usage. On average, the sharder’s RSS is about 89% of the singleton’s RSS. The other two shards use significantly less memory: on average, roughly 40% of the singleton.

Finally, fig. 11 compares the used network bandwidth of both setups by direction. It shows that the amount of transmitted bytes is very similar in each instance of the sharded setup, while the sharder receives significantly more bytes than the other two shards. The total bytes received by the sharder are 89% of the singleton’s total, while the other two shards receive 42% as many bytes as the singleton. The sharded operator instances transmit about 44% as many bytes as the singleton.

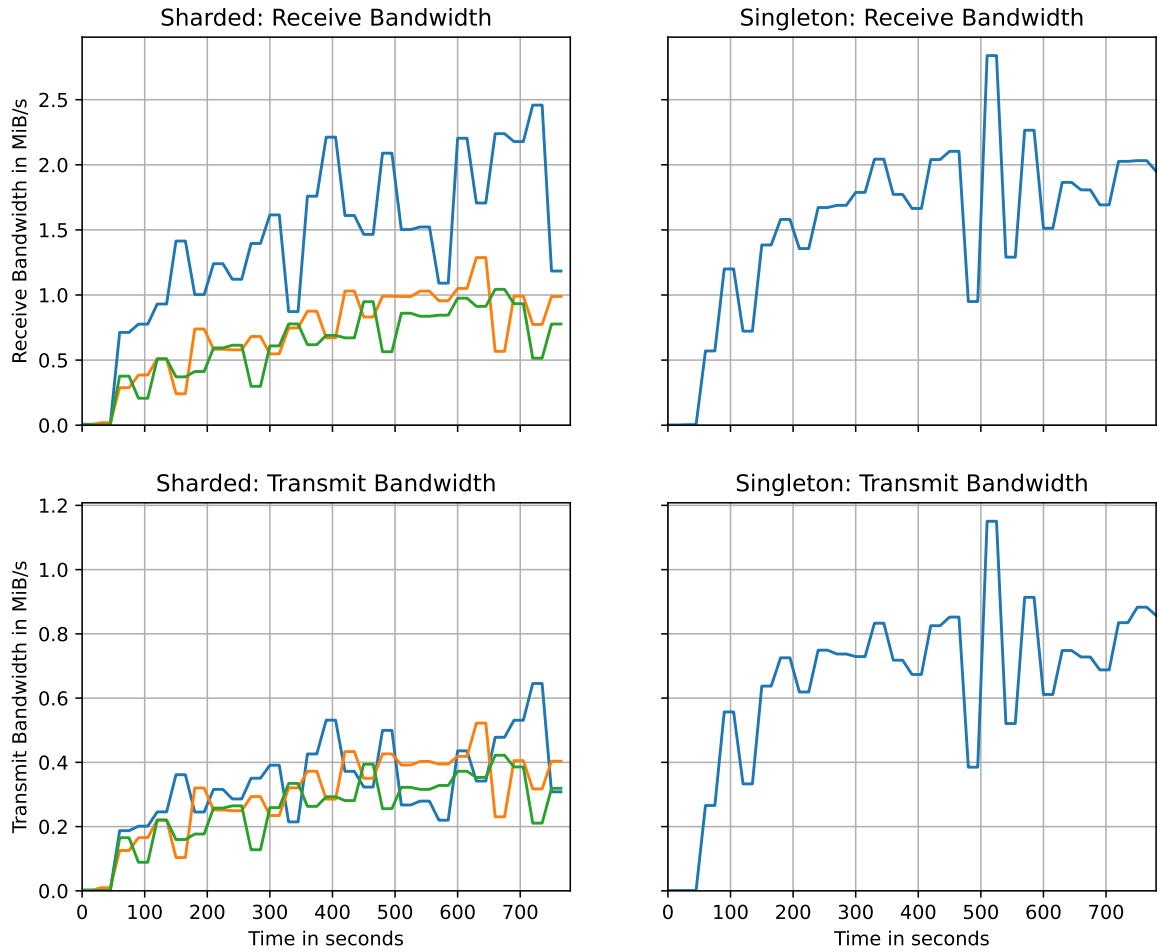


Figure 11: Network bandwidth by pod

6.5 Discussion

The results presented in the previous section demonstrate that the proposed design for sharding in Kubernetes controllers achieves a distribution of the required resources across controller instances. First of all, the implemented mechanisms realize a good distribution of responsibility for API objects among the three shards. It is shown that the individual shards of the system require roughly half as many resources in comparison to the singleton controller. With that, the presented sharding mechanism overcomes the current scalability limitation of Kubernetes controllers and makes them horizontally scalable. Using such an architecture allows increasing the system's capacity and throughput by adding more controller instances.

However, the results also highlight that the instance that is the active sharder consumes almost as many resources as the singleton controller. This limits the scalability of the system again by the available machine sizes and network bandwidth similar to the singleton controller architecture. Hence, the scalability limitations described

in sec. 3.1 are not removed entirely. It is important to note, that in the evaluated deployment setup, the sharder runs both the sharder and shard components. I.e., it is responsible for assigning objects to individual instances as well as running the actual object controllers just like the other instances. This results in significantly higher resource consumption in comparison to the other instances that only run the shard components.

Nevertheless, the presented design does not prescribe that the sharder must also act as a shard at the same time. The deployment setup could be adapted to run the shard and sharder in dedicated instance sets (i.e., **Deployments**). Based on the presented experiment results, it is expected that this results in better distribution of resource requirements. With that, the original scalability limitation in the vertical direction can be overcome entirely and the system's scalability can be increased even further.

The results also show that the two pods that only run the shard components have very similar resource usage. In a typical singleton controller setup with multiple instances for active-passive HA, there is a much more heterogeneous resource usage as the passive instances consume almost no resources in comparison to the leader. With this, the sharded setup also brings advantages for vertical scaling as equally-sized instances can be right-sized more easily to minimize wasting reserved resources.

Last, it is worth noting that the simulated load is still very low in comparison to some production deployments of controllers. With even more and larger API objects in the system, the difference between the sharder's and singleton's resource usage might be more significant.

7 Conclusion and Future Work

This thesis performed the first steps toward horizontally scalable Kubernetes controllers. As part of the motivation (chapter 1), it was shown that the current leader election mechanisms effectively prevent scaling Kubernetes controllers horizontally. Because of this, Kubernetes controllers can only be scaled vertically which restricts their scalability to the available machine sizes and network bandwidth of the underlying infrastructure. After laying out the relevant background of Kubernetes controllers for this thesis, important proven sharding mechanisms of distributed databases were summarized (chapter 2). This served as a basis for analyzing what is required for removing the current scalability limitations of Kubernetes controllers and scaling them horizontally (chapter 3). Based on this analysis, a design was presented that applies well-known sharding mechanisms from the field of distributed databases to the specific problem of sharding in Kubernetes controllers. The proposed design gives an architectural overview and then describes the details of how the listed requirements can be fulfilled (chapter 4). The second half of the thesis described how the proposed design is implemented and evaluated. For this, the webhosting operator was implemented as an example operator for demonstrating the sharding mechanisms in practice. The operator was implemented using the controller-runtime library, which was enhanced to generically realize the presented sharding design (chapter 5). In the last chapter, the implementation was used as a basis for evaluation in an experiment. After describing the deployment and monitoring setup in detail, a load test experiment was conducted to measure how well the sharded operator scales horizontally (chapter 6).

To summarize, this thesis showed that well-known sharding mechanisms from the field of distributed databases can be applied well to Kubernetes controllers. The resulting design allows for overcoming the current scalability limitation of controllers and scaling them horizontally. The conducted evaluation shows that the presented sharding implementation works well and realizes a good distribution of responsibility for objects among multiple controller instances. Furthermore, the resource requirements of the singleton controller are distributed across multiple shards. However, the sharder instance still has a high resource usage and thus faces similar scalability limitations in the vertical direction as the singleton controller setup (sec. 6.4). Nevertheless, the measurements lead to the conclusion that this limitation can be removed by deploying the sharder components separately from the shard components (sec. 6.5).

Apart from this improvement, this thesis lays the ground for future work in several

areas: further optimizations, solving different challenges, evaluation of more aspects, and advanced applications. First, it should be investigated how the use of webhooks for sharded API objects could improve the design and reduce the resource requirements of the sharder components. E.g., a mutating webhook could be used to assign API objects to shards during admission instead of in a dedicated controller. Furthermore, the proposed design faces the challenge that the `shard` and `drain` labels on API objects should not be mutated by humans. If operators grant end users access to the Kubernetes API, validating webhooks could be leveraged to deny user requests that mutate or remove sharding-related labels. Another challenge is, that a single component might run multiple controllers that act on the same object kind but can't share the same sharded cache. In this case, it needs to be investigated how this challenge could be solved by introducing dedicated caches with different selectors. Additionally, some controllers evaluate relationships between objects of the same kind. E.g., the Kubernetes scheduler considers scheduling constraints in form of inter-pod anti-affinities that allow the spreading of related pods across different failure domains. Future work could investigate whether sharding mechanisms can be applied to such use cases as well.

Furthermore, it should be evaluated what impact the presented sharding mechanisms have on the resource requirements and scalability of Kubernetes control plane components, i.e. the Kubernetes API server and etcd (tbl. 2). Also, this thesis only evaluated the resource requirements of a static set of sharded controller instances. Future work could conduct experiments that additionally perform rolling updates of the operator `Deployment` or scale-out/in the operator. The impact of reassignment operations during such changes to the instance set needs to be investigated and possibly further optimizations of the design need to be implemented.

Last, future work could evaluate advanced applications of the presented sharding mechanisms. E.g., the sharding design could be implemented in Kubernetes core components like the controller manager. The Kubernetes core controllers are not based on the controller-runtime library [32], hence they can't reuse the implementation done for this thesis. Nevertheless, the proposed design could still be applied to these controllers as well. Another idea for future work is to investigate how sharded controllers could be scaled up or down automatically based on utilization. E.g., a `HorizontalPodAutoscaler` [4] could be introduced which adds more controller instances when the average queue wait duration exceeds a certain target or removes instances when most of the controller workers are inactive. Furthermore, another advanced application of the presented sharding mechanisms is to make the number of virtual nodes per controller instance configurable and thereby control the expected

distribution of objects. This could be used to implement canary rollout approaches for controllers [33], [34]. I.e., when a new controller version is rolled out, only a small subset of objects is assigned to the new version. The object distribution could then be gradually shifted towards the new version to increase the operator’s confidence in the new version.

To conclude, this thesis laid the ground and performed the first steps toward horizontally scalable Kubernetes controllers. It shows that this can be achieved by applying proven sharding mechanisms used in distributed databases to Kubernetes controllers. Performing sharding for Kubernetes controllers comes with a certain overhead, as any other sharding mechanism does as well. Hence, it is not relevant for small controllers or clusters. However, it proves to be interesting for scaling controllers and clusters further that are already heavily loaded and running into scalability limitations. The presented design and implementation are not ready for production deployments yet, but important next steps have been listed that can be performed to make Kubernetes controller sharding production-ready.

Appendix

The implementation belonging to this thesis (including webhosting operator, monitoring setup, experiment tools, etc.) is available at:

<https://github.com/timebertt/kubernetes-controller-sharding/tree/v1.0>

The sharding implementation in the controller-runtime library is available at:

<https://github.com/timebertt/controller-runtime/tree/v0.12.3-sharding-1>

List of Figures

1	Kubernetes cluster architecture [4]	2
2	Building blocks of a controller [15]	10
3	Kubernetes controller sharding architecture	21
4	Sample website managed by webhosting operator	29
5	Webhosting dashboard	38
6	Sharding dashboard	39
7	Controller-runtime dashboard	39
8	Load generated by base scenario	42
9	CPU usage by pod	43
10	Memory usage by pod	44
11	Network bandwidth by pod	45

List of Tables

1	Resource requirements of a Kubernetes controller	17
2	Resource implications on API server and etcd	17
3	Worker pools of the evaluation cluster	37
4	Experiment resource usage measurements	42

List of Code

1	Example API object	4
2	Example owner reference	5
3	Example CustomResourceDefinition	9
4	Example Lease	13
5	Example webhosting operator objects	28
6	Configuring the controller-runtime manager for sharding	31
7	Setup of a sharded controller	31
8	The ShardedRunnable interface	32
9	Adding shards to the hash ring	34
10	Consistent hashing of object keys	35
11	Wrapping reconciler for sharded controllers (simplified)	36
12	Queries configuration for experiments	40

References

- [1] The etcd Authors, “Etcd documentation.” 2022. Accessed: Jul. 03, 2022. [Online]. Available: <https://etcd.io/docs/v3.5/>
- [2] E. A. Brewer, “Towards robust distributed systems,” in *PODC*, 2000, vol. 7, pp. 343477–343502.
- [3] W. Vogels, “Eventually consistent: Building reliable distributed systems at a worldwide scale demands trade-offs? Between consistency and availability.” *Queue*, vol. 6, no. 6, pp. 14–19, 2008.
- [4] The Kubernetes Authors, “Kubernetes documentation.” 2022. Accessed: Jun. 28, 2022. [Online]. Available: <https://kubernetes.io/docs/>
- [5] K. S. Ahluwalia and A. Jain, “High availability design patterns,” in *Proceedings of the 2006 conference on pattern languages of programs*, 2006, pp. 1–9.
- [6] A. B. Bondi, “Characteristics of scalability and their impact on performance,” in *Proceedings of the 2nd international workshop on software and performance*, 2000, pp. 195–203.
- [7] P. Jogalekar and M. Woodside, “Evaluating the scalability of distributed systems,” *IEEE Transactions on parallel and distributed systems*, vol. 11, no. 6, pp. 589–603, 2000.
- [8] D. J. Abadi, “Data management in the cloud: Limitations and opportunities.” *IEEE Data Eng. Bull.*, vol. 32, no. 1, pp. 3–12, 2009.
- [9] S. Agrawal, V. Narasayya, and B. Yang, “Integrating vertical and horizontal partitioning into automated physical database design,” in *Proceedings of the 2004 ACM SIGMOD international conference on management of data*, 2004, pp. 359–370.
- [10] S. Soltesz, H. Pötzl, M. E. Fiuczynski, A. Bavier, and L. Peterson, “Container-based operating system virtualization: A scalable, high-performance alternative to hypervisors,” in *Proceedings of the 2Nd ACM SIGOPS/EuroSys european conference on computer systems 2007*, 2007, pp. 275–287.
- [11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” PhD thesis, University of California, Irvine, 2000.
- [12] M. Hausenblas and S. Schimanski, *Programming kubernetes: Developing cloud-native applications*. O’Reilly Media, 2019.

- [13] The Kubernetes Authors, “Kubernetes design proposals archive.” 2022. Accessed: Jul. 04, 2022. [Online]. Available: <https://github.com/kubernetes/design-proposals-archive>
- [14] The Kubernetes Authors, “Kubernetes community.” 2022. Accessed: Jul. 05, 2022. [Online]. Available: <https://github.com/kubernetes/community>
- [15] The Kubernetes Authors, “Kubernetes sample controller.” 2022. Accessed: Jul. 06, 2022. [Online]. Available: <https://github.com/kubernetes/sample-controller>
- [16] The Apache Software Foundation, “Cassandra documentation.” 2022. Accessed: Aug. 22, 2022. [Online]. Available: <https://cassandra.apache.org/doc/4.1/>
- [17] Cockroach Labs, “CockroachDB documentation.” Accessed: Nov. 12, 2022. [Online]. Available: <https://www.cockroachlabs.com/docs>
- [18] F. Chang *et al.*, “Bigtable: A distributed storage system for structured data,” in *7th USENIX symposium on operating systems design and implementation (OSDI)*, 2006, pp. 205–218.
- [19] M. Burrows, “The chubby lock service for loosely-coupled distributed systems,” 2006.
- [20] J. C. Corbett *et al.*, “Spanner: Google’s globally distributed database,” *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 1–22, 2013.
- [21] MongoDB, Inc., “MongoDB documentation.” 2022. Accessed: Aug. 23, 2022. [Online]. Available: <https://www.mongodb.com/docs/>
- [22] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, “Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web,” in *Proceedings of the twenty-ninth annual ACM symposium on theory of computing*, 1997, pp. 654–663.
- [23] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, “Chord: A scalable peer-to-peer lookup service for internet applications,” in *Proceedings of the 2001 conference on applications, technologies, architectures, and protocols for computer communications*, 2001, pp. 149–160.
- [24] G. DeCandia *et al.*, “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of twenty-first ACM SIGOPS symposium on operating systems principles*, 2007, pp. 205–220.
- [25] The Kubernetes Authors, “The kubebuilder book.” 2022. Accessed: Nov. 23, 2022. [Online]. Available: <https://book.kubebuilder.io/>

- [26] The Kubernetes Authors, “Kubernetes controller-runtime project.” 2022. Accessed: Nov. 23, 2022. [Online]. Available: <https://github.com/kubernetes-sigs/controller-runtime>
- [27] Google Inc., “groupcache.” Accessed: Oct. 11, 2022. [Online]. Available: <https://github.com/golang/groupcache>
- [28] The Kyverno Authors, “Kyverno documentation.” 2022. Accessed: Nov. 23, 2022. [Online]. Available: <https://kyverno.io/docs/>
- [29] Prometheus Authors, “Prometheus documentation.” 2022. Accessed: Oct. 25, 2022. [Online]. Available: <https://prometheus.io/docs/>
- [30] Grafana Labs, “Grafana documentation.” 2022. Accessed: Oct. 25, 2022. [Online]. Available: <https://grafana.com/docs/grafana/latest/>
- [31] The kernel development community, “The linux kernel documentation.” Accessed: Nov. 06, 2022. [Online]. Available: <https://www.kernel.org/doc/html/latest/>
- [32] The Kubernetes Authors, “Kubernetes.” 2022. Accessed: Jun. 28, 2022. [Online]. Available: <https://github.com/kubernetes/kubernetes>
- [33] G. Schermann, J. Cito, and P. Leitner, “Continuous experimentation: Challenges, implementation techniques, and current research,” *IEEE Software*, vol. 35, no. 2, pp. 26–31, 2018.
- [34] B. Adams, S. Bellomo, C. Bird, T. Marshall-Keim, F. Khomh, and K. Moir, “The practice and future of release engineering: A roundtable with three release engineers,” *IEEE Software*, vol. 32, no. 2, pp. 42–49, 2015.